République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université de Tunis El Manar
Ecole Nationale d'Ingénieurs de Tunis

Thesis

submitted in partial fulfillment of the requirements of the

Research Master's Degree

in **Information Processing and Complexity of The Living**

by

Aymen Bahri

Electrical Engineer ENIM 2021

Synthesis of Realistic Electrocardiograms and R-peak Detection using Deep Learning

Defended on the 09th of December 2023 before the jury:

Chairman	:	Wady Naanaa	Professor at ENIT
Reviewer	:	Soumaya Meherzi	Assistant Professor at ENIT
Company supervisor	:	Maroua Mehri	Research supervisor
Academic supervisor	:	Sonia Djaziri Larbi	Associate Professor at ENIT

Research Laboratory /Company

Epsidy

Academic year 2022/2023

Dedication

I would like to dedicate this work to my parents. for whom I am eternally grateful...

Acknowledgments

First of all, I would like to express my most sincere appreciation and thanks to my academic Advisors, **Prof. Sonia Djaziri Larbi**,**Prof. Maroua Mehri** for their guidance and constant encouragement throughout my six months of my master's project. their help and guidance have made my work a very pleasant and fulfilling one.

I would like to thank , **Guillaume Calmon**, and **Myline Cottance** for offering me the opportunity to discover the field of cardiac medicine,

To my friends, for supporting me no matter what, for being my family away from home, I couldn't do any of this without your help and love.

The biggest thanks to my parents and sister for always believing in my potential, for understanding my frustrations and for lifting me up when I always needed.

Last but not least, I would like to express my thanks to my master's professors at the National Engineering School of Tunis, prof. **Linda Marrakchi Kacem**, prof. **Rim Amara**, prof. **Farouk Mhamdi**, prof **Zouhour Ben Azzouz** and prof. **Nicolas Lomenie** for always inspiring and motivating me.

Abstract

The electrocardiogram **(ECG)** which represents the heart electrical activity, is widely used in clinical practice, e.g. during magnetic resonance imaging (MRI) examinations, **cardiac MRI**.In cardiac MRI, the ECG is used to compensate the motion of the heart muscle. This is achieved by synchronizing the acquisition of the MRI data with heart motion, which can be obtained from the **ECG** signal by QRS detection **(gating)**.

The static magnetic field and the switched gradient magnetic fields of an MRI machine severely distort the ECG signals acquired inside the MRI machine (**Magneto-Hydrodynamic (MHD) effect**), which makes the QRS detection a difficult operation.

Deep Learning (**DL**) was widely used in a variety of fields (Computer Vision, Natural Language Processing, etc. . .) and proved its superiority in many applications, e.g., Generative Pre-trained Transformer(GPT).

Such DL models require a large amount of data to achieve great performances, and the availability of such data is a challenge, especially in the medical field.

In this Master's project, we will be interested in the generation of realistic **ECG's** to tackle the problem of the luck and unavailability of sufficient public datasets in the medical field, (heart activity signals) a **D**eep **C**onvolutional **G**enerative **A**dversarial **N**etwork (**DCGAN**), a **R**uccerent **N**eural **N**etwork **GAN**(**RNN-GAN**), a **D**eep **C**onvolutional **V**araitional **A**uto **E**ncoder (**DC-VAE**), and an **RNN-VAE** were implemented and compared for generating ECG's.

We will augment the main dataset with the generated ECG and train **QRS-complex** detection deep learning algorithms, where I will compare a **Mini-Xception UNet-like** model, **Transformer Encoder** based model and the **UNet** reference model that Epsidy uses.

Contents

Acknowledgments **ii**

Abstract **iii**

Introduction **x**

1 Theoretical Concepts **1**
1.1 Introduction to heart functioning and electrography 1
1.1.1 The cardiac muscle 2
1.1.2 Heart conduction system 5
1.1.3 Electrocardiograms(ECGs) 6
1.2 Cardiac MRI, magneto-hydrodynamic effect and gating 9
1.2.1 Basic MRI physics 9
1.2.2 The importance of an ECG during cardiac MRI 15
1.2.3 Magneto-hydrodynamic effect 16
1.3 Conclusion 18

2 Fundamentals of deep learning **19**
2.1 Introduction 19
2.2 Machine learning 19
2.2.1 Deep learning 20
2.2.2 Supervised learning and optimization scheme 23
2.2.3 Back propagation 24
2.3 Convolutional and recurrent neural networks 27
2.3.1 Convolutional neural networks 27
2.3.2 Recurrent neural networks 29
2.4 Deep learning for data generation 30
2.4.1 Auto-Encoders 30
2.4.2 Variational auto encoders VAEs 31
2.4.3 Generative adversarial networks(GANs) 32
2.4.4 Maximum mean discrepancy MMD 33
2.5 The Transformer architecture 35
2.5.1 The attention mechanism (self attention) 35

2.5.2 Positional Encoding . . . 37
2.5.3 Transformer Encoder . . . 39
2.6 R-Peak detection as a segmentation problem . . . 40
2.6.1 Types of segmentation . . . 40
2.6.2 U-net architecture . . . 41
2.6.3 Mini-Xception U-Net like . . . 41
2.6.4 U-net like transformer-encoder based network . . . 42
2.6.5 Model validation . . . 42
2.7 Conclusion . . . 42

3 Realistic Data generation and R peak detection 43
3.1 Introduction . . . 43
3.2 Dataset description and dataset scaling . . . 43
3.3 Generative models . . . 46
3.3.1 CNN-CNN and LSTM-LSTM VAEs . . . 46
3.3.2 CNN-CNN and LSTM-LSTM GANs . . . 47
3.4 Data generation . . . 49
3.4.1 Generating with a CNN-CNN VAE . . . 49
3.4.2 Generating with an LSTM-LSTM VAE . . . 50
3.4.3 Generating with an CNN-CNN GAN . . . 51
3.4.4 Generating with an LSTM-LSTM GAN . . . 54
3.5 R-Peak detection without data augmentation . . . 56
3.5.1 Epsidy's U-Net model results . . . 57
3.5.2 Mini-Xception U-Net like model . . . 60
3.5.3 Transformer based model . . . 61
3.6 R-peak detection with data-augmentation . . . 62
3.6.1 Data augmentation with an LSTM-LSTM VAE . . . 62
3.6.2 Data augmentation with a CNN-CNN GAN . . . 63
3.7 Conclusion . . . 63

4 Conclusion and prospects 65

List of Figures

1.1 Cross-section of the heart muscle (source[Encyclopedia Britannica]) 1
1.2 Cardiomyocytes 2
1.3 Sodium potasium pump 3
1.4 Membrane charge at the resting potential (Source [1]) 3
1.5 Voltage evolution within a cardiomyocite in response to the ionic fluxes . . . 4
1.6 Sinoatrial Node [2] 5
1.7 Autonomic nervous system. (source[[3]] 6
1.8 A complete ECG Cycle [2] 7
1.9 heart's electrical system [2] 7
1.10 the electrical field vector orientation (cardiac vector) [2] 8
1.11 an example of electrode positioning(3 leads) 8
1.12 12 leads configuration 9
1.13 Alignment of hydrogen nucleus with an applied static magnetic field (source[4] 11
1.14 Precession of the magnetic moment around B_0 12
1.15 The electromagnetic spectrum 13
1.16 Energy transfer during excitation 13
1.17 Flip angle 14
1.18 The MR signal 14
1.19 NVM before and after the application of the RF signal 15
1.20 RF pulses and the received signal 16
1.21 ECG gated sequence where six slices are aquired 16
1.22 non-gated and gated thorax MR images 17
1.23 ECG inside and outside MR environment (source [5]) 18

2.1 classical programming and machine learning (source [6]) 20
2.2 A single layer neural network 21
2.3 sigmoid function 21
2.4 ReLU function 22
2.5 neural network (source [6]) 23
2.6 loss function [6] 24
2.7 optimization [6] 25
2.8 1D loss curve [6] 26

2.9 the decomposition of a cat face into a hirarchy of visual modules [6] 28
2.10 Convolution operation over an image [7] 28
2.11 a recurrent network [6] 29
2.12 a simple RNN with 3 units [6] 30
2.13 auto-encoder 31
2.14 variational auto-encoder 32
2.15 the generator is trained to fool the discrimiator [6] 33
2.16 the transformer architecture [8] 36
2.17 attention [6] 37
2.18 self attention 38
2.19 a position encoding vector of a 512 ECG signal 39
2.20 transformer encoder 39
2.21 an example of a mask 40
2.22 Unet Architecture [9] 41

3.1 slicing data 44
3.2 scaling data 44
3.3 12 leads 512-timesteps slices 45
3.4 Generative models 46
3.5 CNN encoder 47
3.6 CNN decoder 47
3.7 RNN discriminator 48
3.8 RNN generator 48
3.9 generated data on epoch 0 49
3.10 generated data at epoch 30 50
3.11 train data distribution on the latent space 51
3.12 generated leads 52
3.13 Generated leads using LSTM-LSTM VAE 53
3.14 data projected on the latent space 53
3.15 generated leads with an LSTM-LSTM VAE 54
3.16 Generated leads using CNN-CNN GAN 55
3.17 Generated leads using CNN-CNN GAN 55
3.18 generator and discriminator losses 56
3.19 a random generated sample 57
3.20 Generated leads using LSTM-LSTM GAN 58
3.21 a random sample from DIJON dataset 60

List of Tables

3.1 LSTM-LSTM and CNN-CNN VAEs trainable parameters 51
3.2 recording files 58
3.3 number of trainable parameters of each model 59
3.4 Epsidy's Model results 59
3.5 Epsidy's Model DIJON test results 59
3.6 Mini-Xception Model train results 60
3.7 Mini-Xception U-Net like Model DIJON test results 61
3.8 Transformer based model, DIJON test results 61
3.9 Transformer based Model, DIJON test results 61
3.10 Transformer based model LOOCV results with reduced dataset size 62
3.11 Transformer based model test results on Dijon dataset 62
3.12 LOOCV results with data augmentation using an LSTM-LSTM VAE 63
3.13 Test on Dijon dataset with VAE data augmentation 63
3.14 LOOCV results with data augmentation using an CNN-CNN GAN 64

Nomenclature

AV AtrioVentricular Node

$\mathbf{B_0}$ static magnetic flux density inside an MR machine

CNN Convolutuional Neural Network

ECG ElectroCardioGram

GAN Generative Advarsail Network

MHD Magneto HydroDynamic

MMD Maximum Mean Discrepancy

NVM Net Magnetic Vector

RF Radio Frequency

RNN Recuurent Neural Network

SA Sinoatrial Node

VAE Variational Auto-Encoder

Introduction

Magnetic resonance imaging (MRI) is a medical imaging technique used to study the anatomy and physiology of the body. It has been applied for a wide range of diagnostic purposes including cancer, cardiology, vascular diseases, neurology and others.

One very important application of MRI is cardiovascular magnetic resonance (CMR). CMR covers the anatomical and functional diagnose of the heart muscle and related blood vessels. CMR applications – either diagnostic or therapeutic – require an electrocardiogram (ECG) of the patient. The ECG represents the electrical activity of the heart muscle or myocardium and is acquired by electrodes placed on the surface of the thorax. For MRI, a trace of the ECG can be required for:

- synchronizing the acquisition of MRI data with heart motion (gating)
- monitoring the patient's vital parameters.

For a normal ECG signal, the QRS complex has the highest amplitude peak and sharpest upstroke, which is often used for gating.

However, the ECG based cardiac gating is associated with several potential issues. First, the ECG signal is sometimes interfered by the static field of the MRI system, (**The Magneto-Hydrodynamic (MHD) effect**). Such interferences can be severe at higher fields and eventually cause degraded image quality due to synchronization errors.

classical algorithms were proposed for QRS complex detection, such algorithms are based on handcrafted rules, which makes them vulnerable to acquisition environments change, contrary, Deep Learning models learn those features from data, by exposing them to a large amount of data, DL models can generalize to difficult tasks, such as QRS detection in 3T,7T MRI environments.

The limited availability of training data remains a major challenge, particularly in the medical field, where data acquisition can be costly and subject to privacy regulations.

Data augmentation techniques offer a solution by artificially increasing the number of training samples, but these techniques often produce limited and unconvincing results.

To address this issue, a growing number of studies have proposed the use of deep generative models to generate more realistic and diverse data that conform to the true distribution of the data, such as Generative Adversarial Networks(GANs) and Variational autoencoders (VAEs).

The report will be structured as follows:

- **Chapter 1** Reviews the main functioning of the heart muscle, acquisition of ECGs and why the latter is required during cardiac MRI, and briefly describes the magneto-hydrodynamic effects, and R-peak detection becomes difficult inside MR machines
- **Chapter 2** Reviews machine learning and deep learning, briefly outlines variational auto-encoders and generative adversarial networks, and the transformer architecture.
- **Chapter 3** Presented the main work done in the internship, mainly the data generation part and the R-peak detection part.

Chapter 1

Theoretical Concepts

1.1 Introduction to heart functioning and electrography

The central function of the heart is to contract rhythmically and pump blood to the lungs for oxygenation, and then to pump this oxygen-enriched blood into the general (systemic) circulation.
The signal that initiates the cardiac contraction is the spread of electrical currents through the heart muscle. These currents are produced **both** by **pacemaker cells**, **specialized conduction tissue** within the heart, and by the working **heart muscle** itself, i.e, the generation of electrical currents through the heart doesn't have a single source.

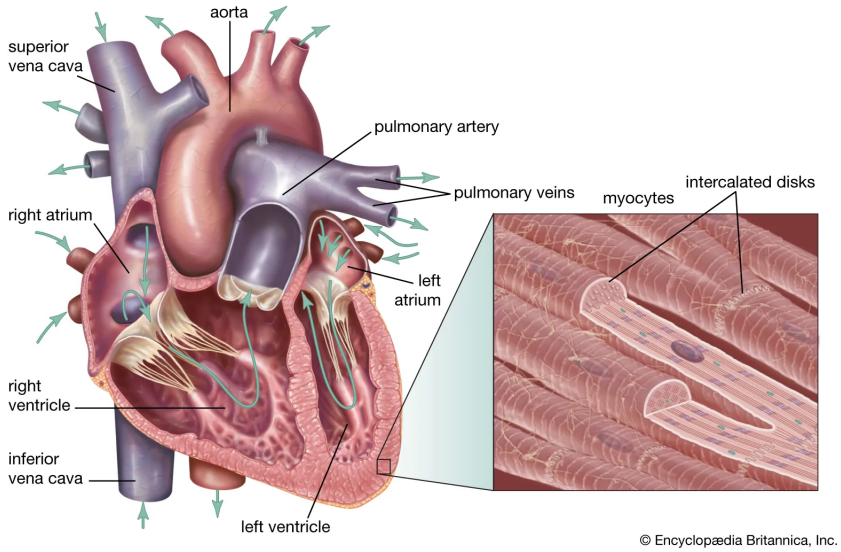


Figure 1.1: Cross-section of the heart muscle (source[Encyclopedia Britannica])

1.1.1 The cardiac muscle

The cardiac muscle is composed of two main cell types: **Cardiomyocytes**, which generate electrical potentials during contraction, and **other** type of cells specialized in the generation and conduction of the action potentials.

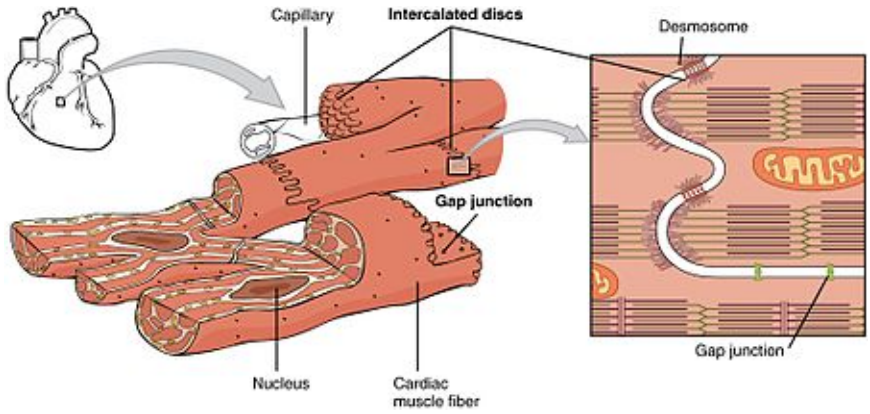


Figure 1.2: Cardiomyocytes

Cardiomyocites, like other cells in the body, are bounded by a lipoprotein membrane which acts as a good electrical insulator. Across this membrane, between the interior of the cell and the extracellular fluid, there is usually an electrical potential difference, the membrane potential. This potential influences the processes of exchange through the membrane, and in this respect is important.

Cardiomyocites can be polarized and depolarized by a change in their membrane potential, this change is due to a flux of ions that move inside and outside the cell membrane :
The membranes divide the medium into extracellular and intracellular fluids, which have different ionic concentrations.
Potassium (K+), sodium (Na+), calcium (Ca2+) and chlorine (Cl–) ions and other protein anions (noted A-) are involved in the creation of the cell potential.

Generally in muscle and nerve cells, the membrane potential remains constant for long periods if the cells are not activated by an external influence, the membrane potential of such resting cells is called the **resting potential**.

The resting potential of nerve and muscle cells is always negative,its magnitude is constant and characteristic of each cell type and for Cardiomyocites is about $-90mV$ [1].

Active sodium-potassium pumps, which are located in the membrane of the cells (fig 1.3), actively expel Na+ from the cell and move K+ into the cell and vice versa, this results in an enrichment or depletion (respectively depletion and enrichment of Na+ concentration inside and outside the cell).

figure 1.3 :

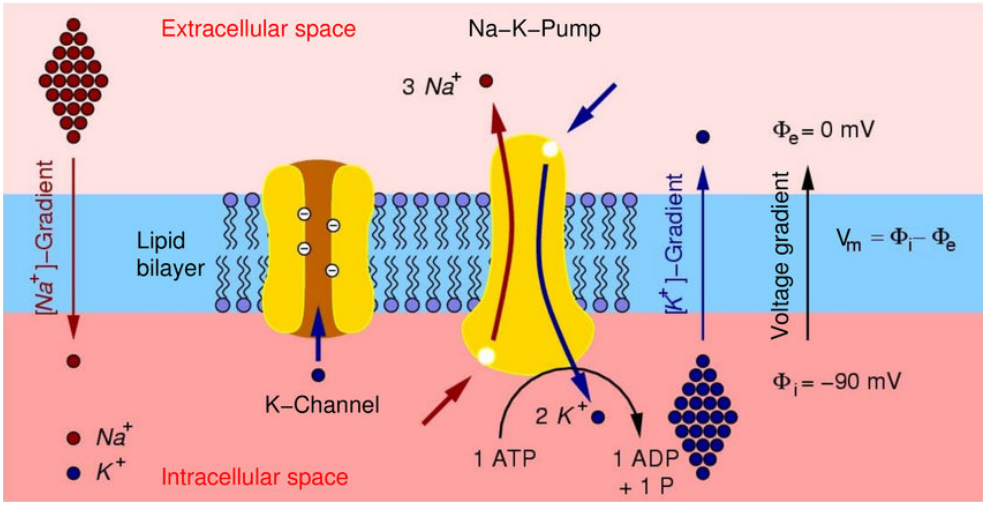


Figure 1.3: Sodium potasium pump

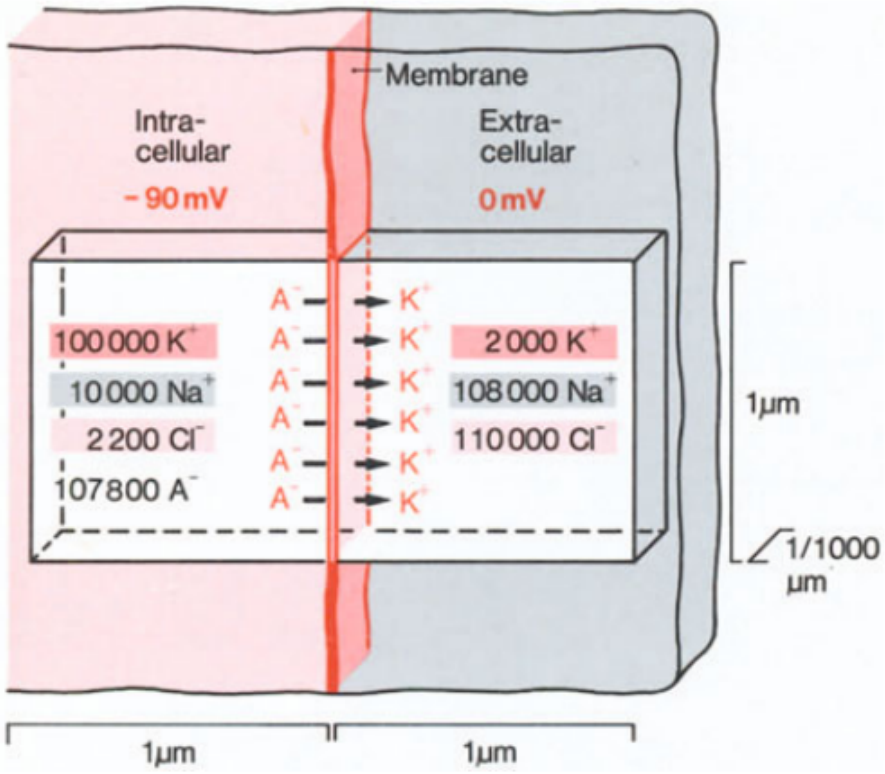


Figure 1.4: Membrane charge at the resting potential (Source [1])

During the **depolarization** or **excitation** of a cell, its membrane permeability is modified with respect to certain ions. Firstly, the permeability to sodium (Na+) is increased, hence, Na+ ions flow from the extracellular to the intracellular space which causes the mem-

brane potential to rise up to 30mV,in this stage, the cell is said to be depolarized.
at this point Na+ channels are closed and Ca2+ channels are open which enables calcium ions to flow slowly into the cell, This Ca2+ influx slightly **repolarizes** the cell and causes the characteristic plateau phase which is depicted in Fig. 1.5.

Now Ca2+ channels are closed and Potassium(K+) channels are reopened which causes a flow of (K+) into the cell and a (Na+) ions out of the cell, and in this stage the cell is said to be **repolarized**.

The flux of ions is a flux of charged particles and with different concentrations of ions inside and outside the cell membrane means a time variant net charge, and we know from the electromagnetic theory that the space displacement of a charged particle with a time variant net charge, makes an electric field that varies through space and time, so we get as a result an electric potential that varies through time. this potential is known as **action potential**.

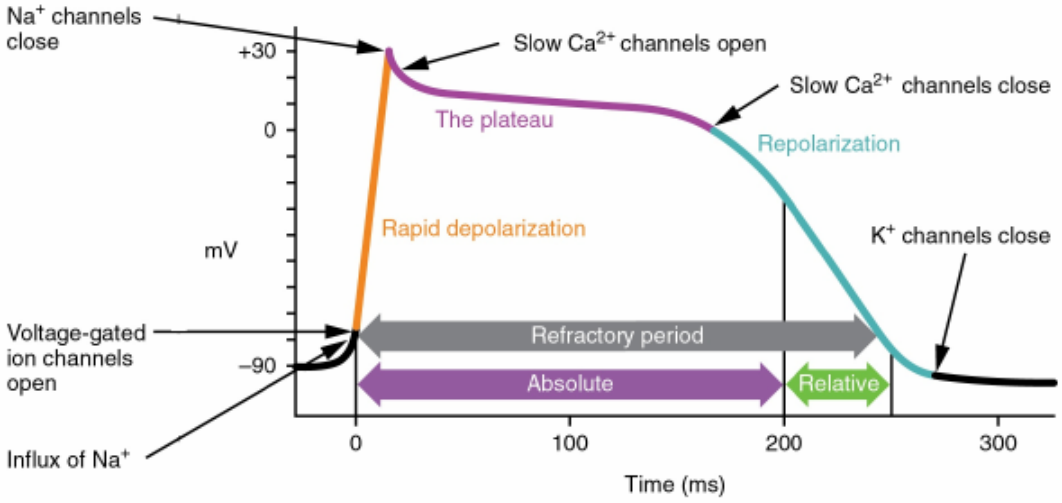


Figure 1.5: Voltage evolution within a cardiomyocite in response to the ionic fluxes

The depolarization of one cell propagates to the adjacent cells through gap junctions, which finally depolarizes the entire cardiac muscle tissue.

The Ca2+ influx causes a mechanical response: the muscle tissue contracts shortly after it was depolarized. The depolarization lasts about 2 ms and repolarization about 200 ms (including the plateau phase). [1].

To be able to know which part is the main producer of the first "spark" that initiates the functioning of the heart from the parts mentioned above, we need to investigate the **heart conduction system** .

1.1.2 Heart conduction system

The heart conduction system is composed mainly of the Sinoatrial node (SA node), the Atrioventricular node (AV node), the bundle of His and the Purkinje fibers.

Sinoatrial Node : The rhythmic contractions of the heart, which pump blood, occur in response to periodic electrical control pulse sequences that are generated by a natural pacemaker located in the superior wall of the right atrium (a specialized bundle of nerve fibers) called the **sinoatrial node (SA node)**. The SA node controls the heart rate by precisely determining when the muscles of the atria and the ventricles contract. Those pacemaker cells are said to be self-excitatory, which makes them the principal source that initiate action potentials of the heart that can pass through cardiac muscle cells and cause contraction.

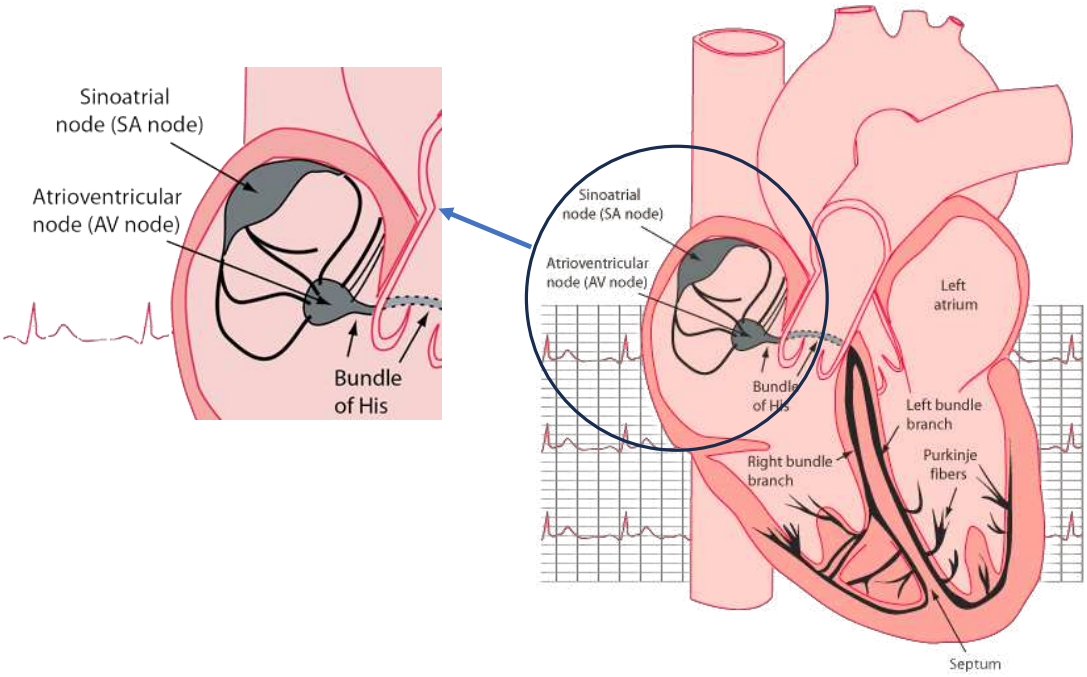


Figure 1.6: Sinoatrial Node [2]

The firing of the **SA** node sends out an electrical impulse via its neurons to the right atrium, left atrium, and AV node simultaneously. Since the right atrium is closer to the SA node, it depolarizes first, resulting in pumping action by the right atrium before the left atrium.

The activity of the S-A node is regulated mainly by the autonomic nervous system. An activation of sympathetic fibers causes an increase in the heart rate, while activation of the parasympathetic fibers results in a decrease in this rate. the autonomic nervous system is depicted in fig.1.7.

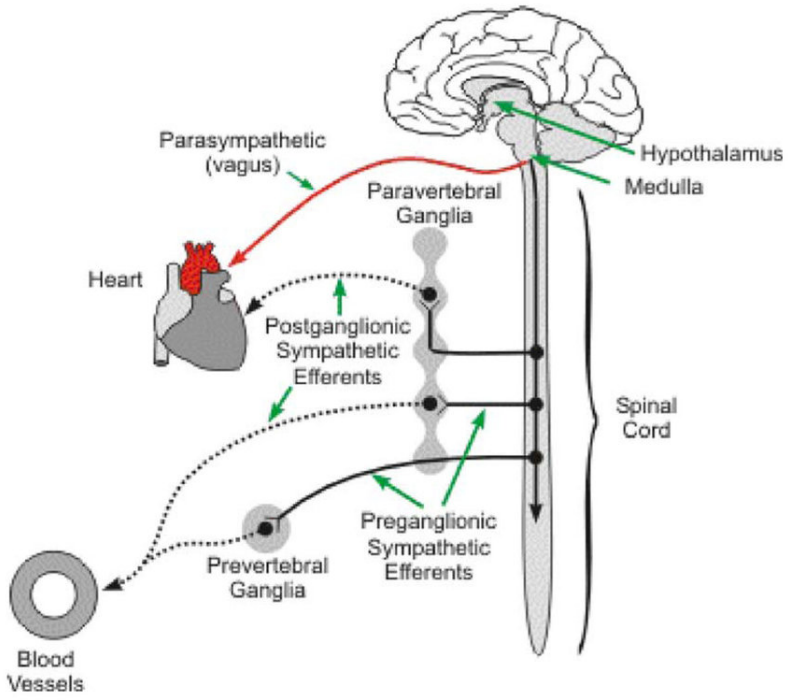


Figure 1.7: Autonomic nervous system. (source[[3]]

The AV node, Bundle of His and Purkinje fibers : At the AV node, the impulse is delayed to allow for the ventricles to fill up with blood. After the delay, the AV node sends the impulse to the Bundle of His and the Purkinje fibers. This triggers the contraction of the ventricles to send blood either to the lungs or out to the body.

1.1.3 Electrocardiograms(ECGs)

The depolarization and repolarization of the SA node and the other elements of the heart's electrical system produce as we discussed above a space-time variant electrical field, which generates in turn a voltage change, that can be measured with electrodes on the skin.
Voltage measurements on the skin of the chest are called an **electrocardiogram, ECG**.
Fig 1.8 shows a complete ECG cycle with a description of each wave meaning.

The signal caused by the depolarization of the SA node propagates concentrically through the right atrium until it reaches the left atrium and the AV node. In the measured ECG signal, the atrial excitation manifests as the P-wave.Fig.1.8
Compared to the propagation velocity within the atrium or ventricles, the velocity through the AV node is relatively slow. The propagation delay prevents a simultaneous contraction of the atria and the ventricles. This enables a complete filling of the ventricles. The propagation delay caused by the AV node can be measured by the ECG's (PQ-PR) interval.
Once the signal passed the AV node and reached the ventricles, it proceeds along the Purkinje

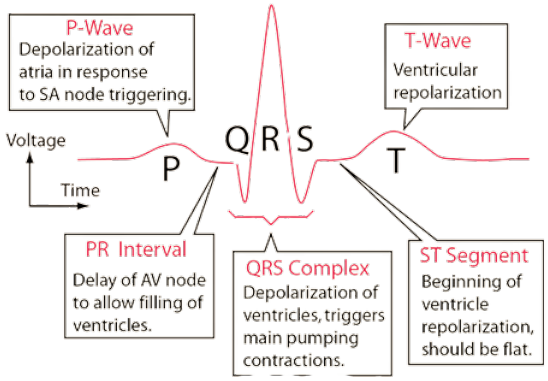


Figure 1.8: A complete ECG Cycle [2]

fibres to the inner walls of the ventricles. The excitation starts from the left ventricle, followed by the activation of the right ventricle, which causes the contraction of the left followed by a construction of the right ventricles. This depolarization is represented by the QRS-complexe of the ECG signal.
When the ventricles are fully excited and electrical current doesn't exist anymore, this phase corresponds to the ST-segment, and it's a manifestation of ventricle repolarization.

Fig 1.9 and 1.10 show further details of the heart conduction system and its corresponding ECG segments.

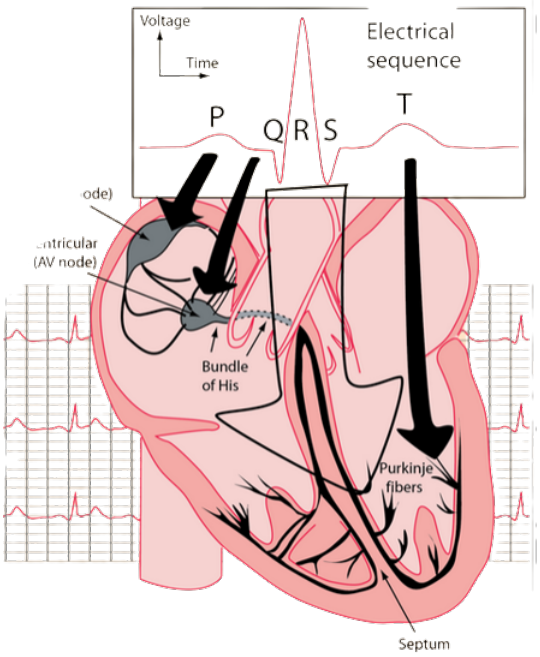


Figure 1.9: heart's electrical system [2]

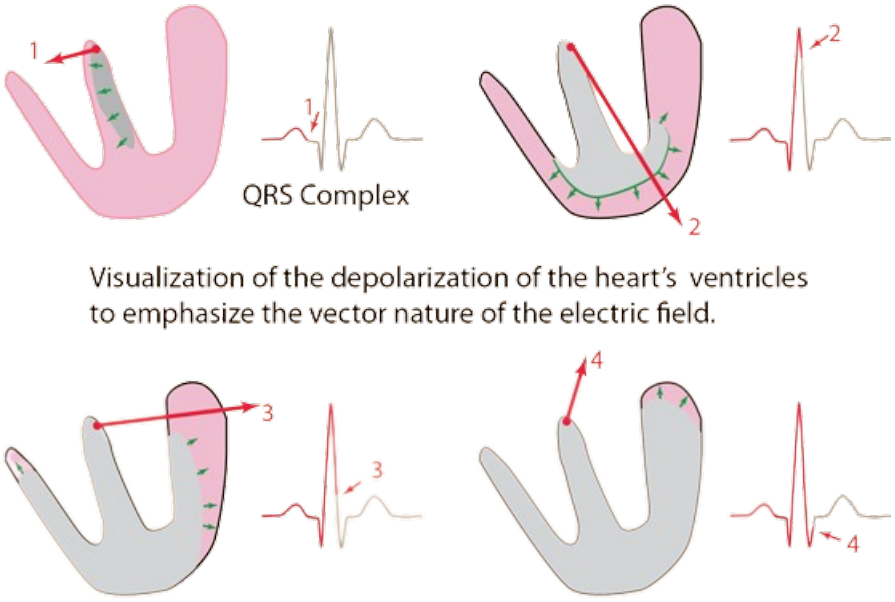


Figure 1.10: the electrical field vector orientation (cardiac vector) [2]

Many different combinations of electrodes have been used clinically, up to twelve leads. From these combinations, the electric field vector representing the heart's electrical activity can be determined as a function of time. But with a set of at least three electrodes, one can measure two components and obtain a value for the magnitude and direction of the field at the surface.

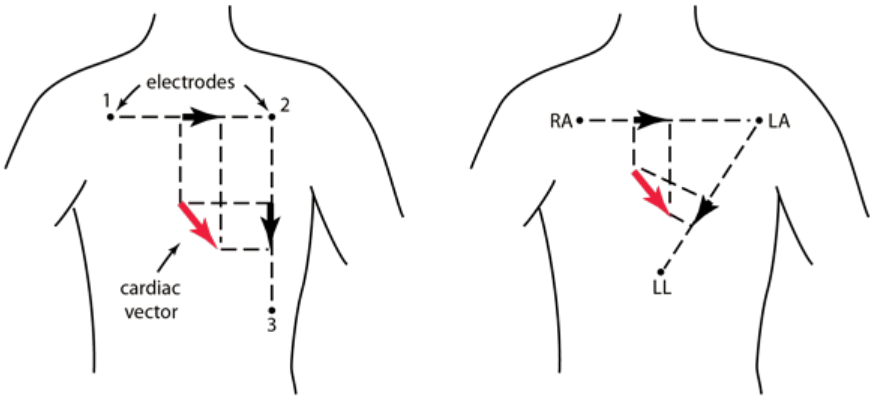
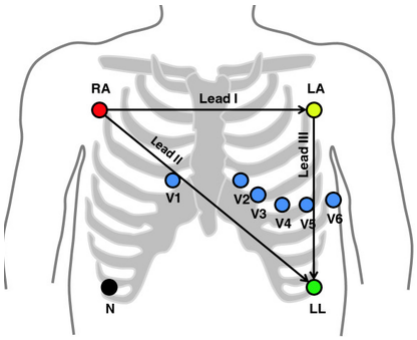
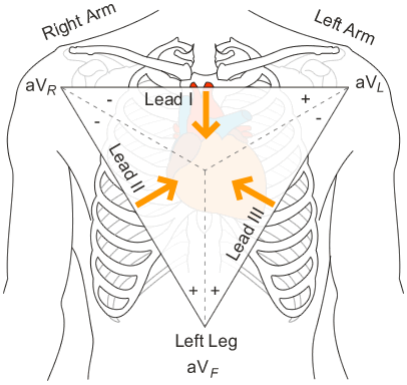


Figure 1.11: an example of electrode positioning(3 leads)

The 12 lead placement is depicted in fig 1.12



(a) 12 leads configuration



(b) 12 leads configuration

Figure 1.12: 12 leads configuration

Each lead is looking to a specific region of the heart :

- aV_L is on the left wrist and looks at the upper left side of the heart.
- Lead I travel towards aV_L creating a second lateral lead.
- aV_F is on the left ankle or left lower abdomen and looks at the inferior wall of the heart.
- Lead II travels from aV_R towards aV_F to become a 2nd inferior lead
- Lead III travels from aV_L towards aV_F to become a 3rd inferior lead.
- V2, V3 and V4 look at the front of the heart and are the anterior leads.
- V1 is often ignored but if changes occur in V3 and V2, these leads are referred to as Septal leads.
- V5 and V6 look on the left side of the heart and are the lateral leads.

1.2 Cardiac MRI, magneto-hydrodynamic effect and gating

1.2.1 Basic MRI physics

All things in nature are made up of atoms, including the human body, and the most abundant atom in the body is hydrogen.
The number of neutrons and protons in a nucleus are usually balanced so that the mass number is an even number. In some atoms, however, there are slightly more or fewer neutrons than protons, this type of atom is important for MRI.

Inside an atom, the nucleus is spinning about its own axis, exactly as an electron does. The principles of MRI rely on the spinning motion of specific nuclei present in biological tissues.

In nuclei that have an even mass number, i.e. the number of protons equals the number of neutrons, half spin in one direction and half in the other. The nucleus itself has no net spin. However, in nuclei with odd mass numbers, i.e. where the number of neutrons is slightly more or less than the number of protons, spin directions are not equal and opposite, so the nucleus itself has a net spin or **angular momentum**. These are known as **MR active nuclei**[4].

MR active nuclei :

MR active nuclei that have a net charge and are spinning (motion), automatically acquire a magnetic moment and can align with an external magnetic field.
Important examples of MR active nuclei are :

- Hydrogen 1
- Carbon 13
- Nitrogen 15
- Oxygen 17
- etc...

The Hydrogen nucleus :

The isotope of the hydrogen nucleus called **protium** is the MR active nucleus used in clinical MRI.
This contains a single proton, and it is used because hydrogen is very abundant in the human body, and also because its solitary proton gives it a relatively large magnetic moment [4]. Both of these characteristics enable the utilization of the maximum amount of available magnetization in the body.

Electromagnetic theory states that a magnetic field is created when a charged particle moves, therefore the hydrogen nucleus has a magnetic field induced around it and acts as a small magnet.

If there is no external magnetic field, the orientation of each magnetic moment of hydrogen atoms nucleus is random, as soon as an external static magnetic field is present, the magnetic moment of the hydrogen nuclei align with this magnetic field, some hydrogen nuclei (magnetic moments) aligns in the same direction as of the external magnetic field, those type of atoms posses low energy, in terms of quantum energy (**spin up nucleus**), and a small number aligns in the opposite direction (**spin down nucleus**), and this type of nucleus possess a high quantum energy.The latter is not important in MRI, since their number is small compared to the atoms that align their magnetic moment in parallel to the

external magnetic field.

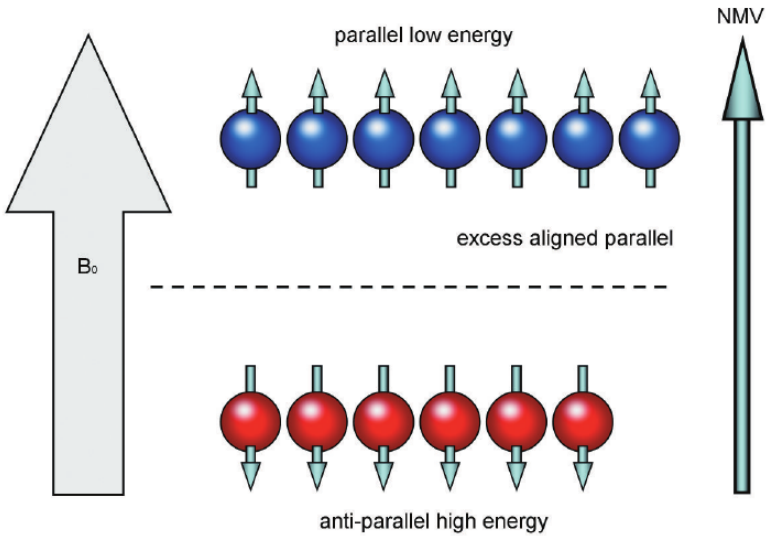


Figure 1.13: Alignment of hydrogen nucleus with an applied static magnetic field (source[4]

The net magnetic moment of hydrogen, is called **the net magnetization vector (NMV)**.

Precession :

The influence of an external static magnetic field B_0 on a hydrogen nucleus that is spinning on its own axis is the production of a secondary spin of the magnetic moment around B_0 called **precession**. Which forces the magnetic moments to follow a circular path around B0(called precessional path).
The speed in which the magnetic moments spin around B_0 is called the precessional frequency.

The value of the precessional frequency is governed by the **Larmor equation**.

$$\omega_0 = B_0 \times \lambda \tag{1.1}$$

Where: ω_0 is the precessional frequency
B_0 is the external magnetic field strength
λ is the gyromagnetic ratio

The gyromagnetic ratio is expressed as the precessional frequency of a specific MR active nucleus at 1 T and for hydrogen $\lambda = 42.57MHz/T$.

Resonance :

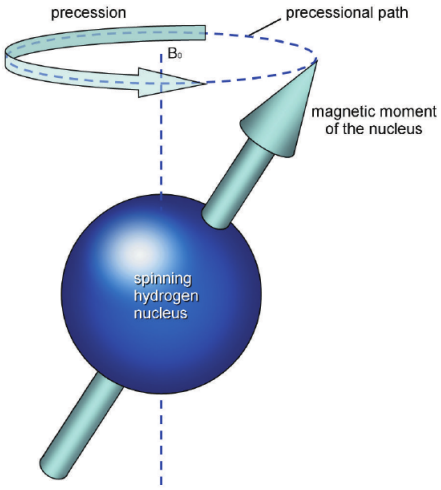


Figure 1.14: Precession of the magnetic moment around B_0

Resonance is a phenomenon that occurs when an object is exposed to an oscillating perturbation that has a frequency close to its own natural frequency of oscillation. When a nucleus is exposed to an external perturbation that has an oscillation similar to its own natural frequency, the nucleus gains energy from the external force and resonates with this frequency.If energy is delivered at a different frequency to that of the Larmor frequency (precessional frequency) of the nucleus, resonance does not occur.

Energy at the precessional frequency of hydrogen at all field strengths in clinical MRI corresponds to the radio frequency (RF) band of the electromagnetic spectrum depicted in figure 1.15.

For resonance of hydrogen to occur, an RF pulse of energy at exactly the Larmor frequency of hydrogen must be applied(**excitation**). Other MR active nuclei that have aligned with B_0 do not resonate, because their precessional frequencies are different to that of hydrogen.

Results of resonance :

One of the results of resonance is that the **NMV** moves out of alignment away from B_0. This occurs because some of the low-energy nuclei(spin-up) are given enough energy via resonance to join the high-energy population (spin-down).

As the NMV reflects the balance between spin-up and spin-down nucleus, resonance causes the NMV to no longer lie parallel to B_0 but at an angle to it called **Flip angle**. The magnitude of the flip angle depends on the amplitude and duration of the RF pulse.

The other result of resonance is that the magnetic moments of hydrogen nuclei move into phase with each other (coherence).

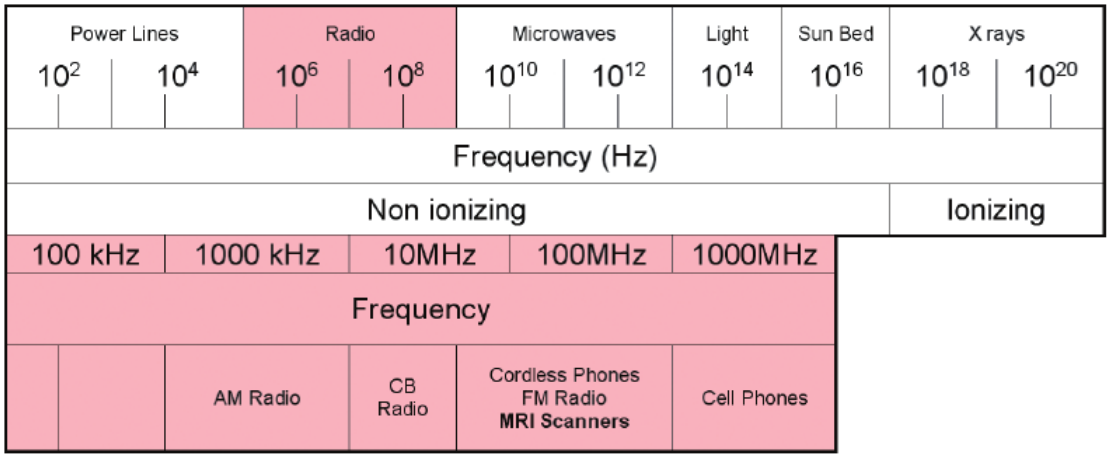


Figure 1.15: The electromagnetic spectrum

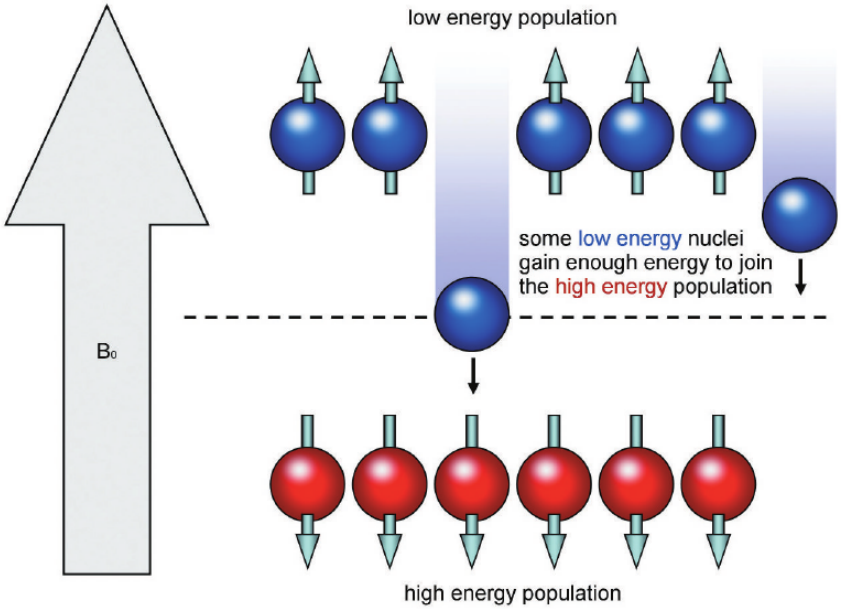


Figure 1.16: Energy transfer during excitation

The MR signal :

As a result of resonance, in phase or coherent magnetization precesses at the Larmor frequency in the transverse plane. Faraday's law of electromagnetic induction states that if a receiver coil or any conductive loop is placed in the area of a moving magnetic field, i.e. the magnetization precessing in the transverse plane (flip angle of 90 degrees, figure 1.17), a voltage is then induced in this receiver coil. The MR signal is produced when coherent (in phase) magnetization cuts across the coil. This voltage constitutes the MR signal that

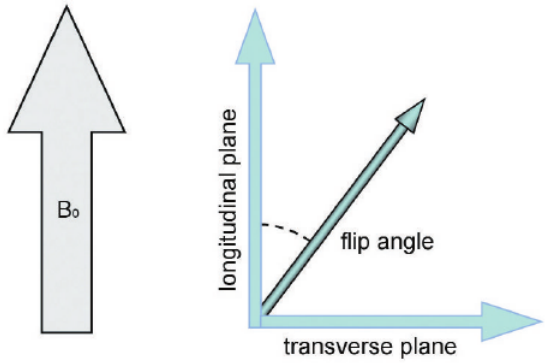


Figure 1.17: Flip angle

oscillates with the Larmor frequency [4].

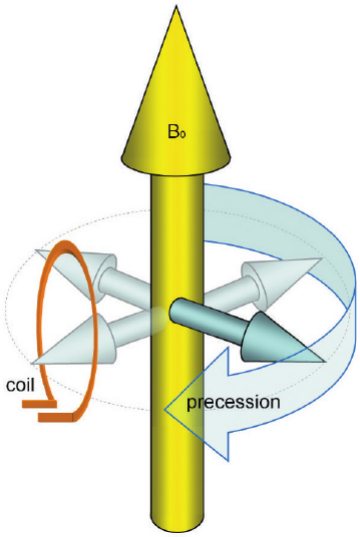


Figure 1.18: The MR signal

The free induction decay signal :

When the RF pulse is switched off, the hydrogen nuclei lose the energy given to them by those pulses (**Relaxation**, the NMV is then influenced only by B_0, and it will try to realign with it, in this process some of the high energy nuclei will return to the spin-up population.

- The amount of magnetization in the longitudinal plane gradually increases (**Recovery**), the process called **T1 recovery**.
- At the same time, the amount of magnetization in the transverse plane gradually decreases (**Decay**), the process is called **T2 decay**.

To summaries, The NMV is a vector quantity. It is created by two components at 90° to each other. These two components are magnetization in the longitudinal plane and magnetization in the transverse plane. Before resonance, there is full longitudinal magnetization parallel to B_0. After the application of the RF pulse and assuming a flip angle of 90°, the NMV is flipped fully into the transverse plane. There is now full transverse magnetization and zero longitudinal magnetization.

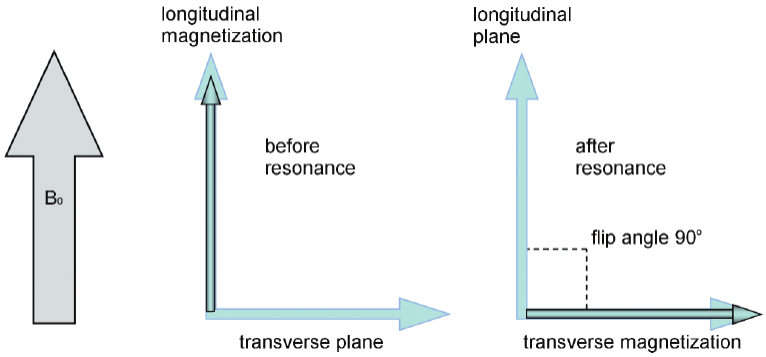


Figure 1.19: NVM before and after the application of the RF signal

Once the RF pulse is removed, the NMV recovers. As this occurs, the longitudinal component of magnetization grows again, while the transverse component decreases. As the received signal amplitude is related to the magnitude of the coherent transverse component, the signal in the coil decays as relaxation takes place.

When we combine RF pulses, we get a pulse sequence, and it's characterized by repetition time (**TR**) and echo time(**TE**), figure.1.20

- The repetition time (TR) is the time from the application of one RF pulse to the application of the next RF pulse.
- The echo time (TE) is the time from the application of the RF pulse to the peak of the signal induced in the coil

The application of RF pulses at certain repetition times and the receiving of signals at predefined echo times produce contrast in MRI images.

1.2.2 The importance of an ECG during cardiac MRI

Acquiring high-quality cardiac MR images of the beating heart remains a technical challenge. Image artifacts can occur due to complex heart motion, respiratory movements, movements of the blood vessels, flow patterns and patient movement [4].

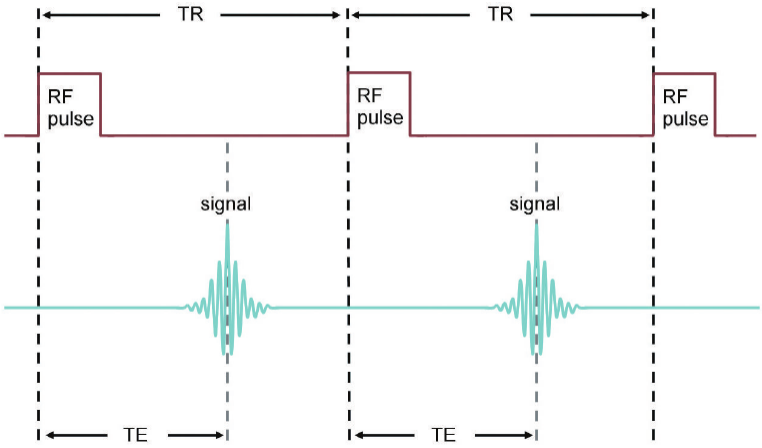


Figure 1.20: RF pulses and the received signal

To image the heart and great vessels specifically, motion during cardiac activity must be compensated to obtain high quality images
A compensation technique known as **cardiac gating** can be used to reduce those unwanted artifacts. Cardiac gating is based on the electrical signal of the heart to trigger the RF pulse of sequences.The RF pulses are initiated by the R wave, which means to be able to obtain high quality images we need to detect R-peak positions.

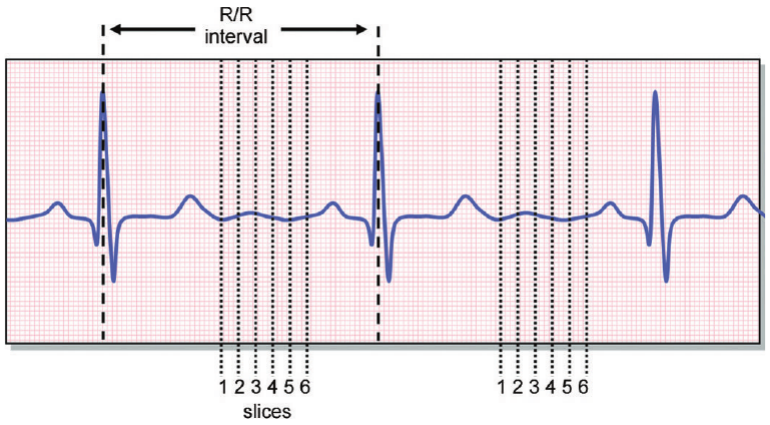
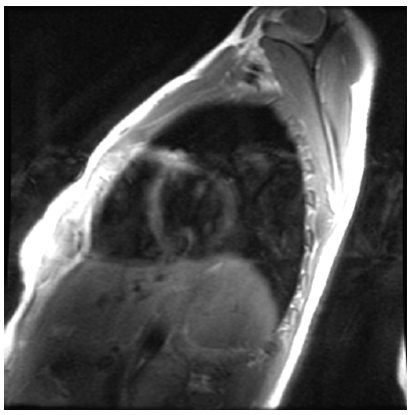


Figure 1.21: ECG gated sequence where six slices are aquired

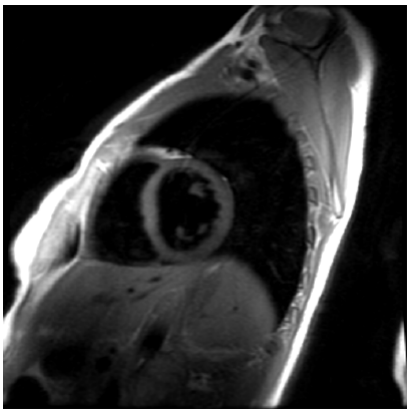
Figure 1.21 shows a gated and a non-gated thorax MR images

1.2.3 Magneto-hydrodynamic effect

Since the blood is an electrical conductive fluid, and according to Friday's law of induction if a conductor moves in a static magnetic field a voltage(e) will be induced in this conductor,



(a) a non-gated MR image



(b) a gated MR image

Figure 1.22: non-gated and gated thorax MR images

i.e, since the blood moves across a magnetic field (the MR machine static magnetic field B_0) a voltage will be induced in the blood.

Faraday's law states that

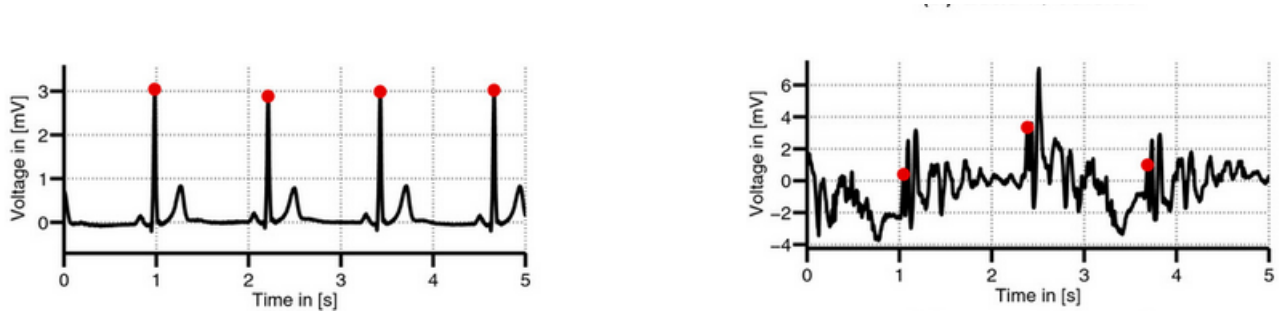
$$e = Blv \tag{1.2}$$

where:
e is the induced voltage
B is the magnetic flux density
l is the length of the conductor, the blood vessels in our case
v is the velocity of the conductor, velocity of blood in our case

What this will cause experimentally is an increase in the amplitude of the T-wave ,and we can see from (equation 1.2) that the higher the magnetic field (magnetic flux density) the higher the induced voltage if blood speed is fixed. This phenomenon is called **magneto-hydrodynamic effect**

The magneto-hydrodynamic effect can present problems when cardiac gating is used, particularly at higher field strengths, and Since the RF pulses are triggered by the R-wave, when the amplitude or height of the T wave increases, the system can trigger from the elevated T wave rather than the R wave. Which is caused by a false detection of the R-wave. The result of inappropriate triggering is degradation in image quality and/or an increase in overall scan time[4].

It's trivial that when ECG are recorded inside an MR environment, R-wave detection becomes a complex problem as the field magnitude becomes higher. Thus, classical algorithms that are based on handcrafted rules cannot generalize better to never seen circumstances, on the other hand machine learning methods are not based on handcrafted rules to learn feature extraction, they learn those rules from data. A machine learning system is trained rather than explicitly programmed, it's presented with many examples relevant to a task,



(a) ECG outside MR environment

(b) ECG inside MR environment

Figure 1.23: ECG inside and outside MR environment (source [5])

and it finds statistical structure in these examples that eventually allows the system to come up with rules.

1.3 Conclusion

In this chapter, I have introduced the main topics needed to understand the internship objectives, the functioning of the heart muscle and electrocardiograms, MRI basic physics and gating, magneto-hydrodynamic effects and the need for deep learning models.

Chapter 2

Fundamentals of deep learning

2.1 Introduction

In this chapter I will be reviewing the building blocks of machine learning and in particular deep learning, first I will briefly describe what is learning in machine learning then briefly introduce deep learning, describe the mathematical building blocks of densely connected layers, review convolutional neural networks and recurrent neural networks, next I will introduce auto-encoders and variational auto-encoders, generative adversarial networks and finely the transformer architecture.

2.2 Machine learning

The difficulties faced by systems relying on hard-coded knowledge suggest that AI systems need the ability to acquire their own knowledge, by extracting patterns from raw data. This capability is known as **machine learning(ML)**. The performance of these simple ML algorithms depends heavily on the representation of the data they are given, in some sense it's related to mathematical statistics, but it differs from statistics in some ways, that is it tend to deal with large, complex datasets (such as millions of images, text data, time series...) for which classical statistical analysis such as Bayesian analysis would be impractical.

Simply, an ML model discovers rules for executing a processing task given examples of what is expected, it transforms its input data into meaningful representations that get it closer to the task at hand, these representations are learned during training, and it needs 3 things:

- Input data
- Examples of the expected output
- A way to measure whether the algorithm is doing a good job or not.

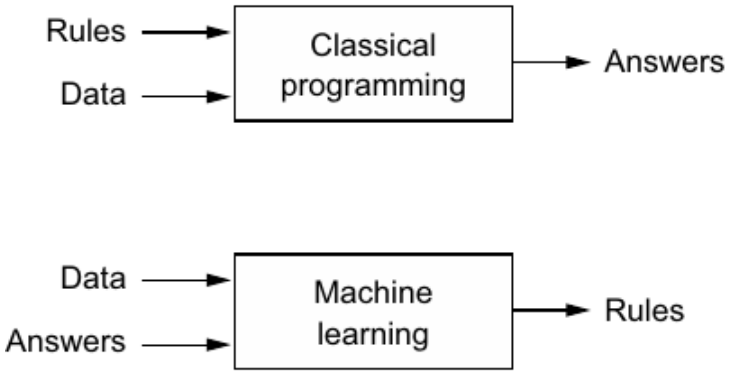


Figure 2.1: classical programming and machine learning (source [6])

Figure 2.1 shows the difference between classical programming and machine learning.

To explain the term learning a representation intuitively, we can check a linear regression algorithm.

If we have an input vector $x \in \mathbb{R}^{n\times 1}$,with elements$\{x_i\}_{i=1}^{n}$, A matrix of trainable weights $W \in \mathbb{R}^{n\times m}$,with elements$\{w_{i,j}\}$, and an output $y \in \mathbb{R}^{m\times 1}$ a linear regression algorithm can be written as(m = 1):

$$f(x) = y = W^T.x \tag{2.1}$$

Now the term $W^T.x$ is a dot product and in algebra it can be interpreted as the projection of x over W , the learning in machine learning is the search for the weights' matrix W ,or in other words for the transformation or **representation** that get the algorithm closer to solve a given task by slightly modifying the weights at each iteration (tweaking the representation to get the best representation of inputs).

2.2.1 Deep learning

Deep learning is a specific subfield of machine learning: a new take on learning representations from data, with learning successive layers of increasingly meaningful representations.The deep in deep learning stands for this idea of successive layers of representation, and it's not a reference to any kind of deeper understanding[6], these layered representation are learned via models called **neural networks**. This term was drawn by inspiration from neurobiology, specifically by our attempt to understand the brain neural mechanism functioning, however There's no evidence that the brain implements anything like the learning mechanisms used in modern deep learning models.

To understand the mathematics behind those model, we can start by understanding the mathematic behind a single layer neural network (also called perceptron).

Fig 2.2 depicts a perceptron

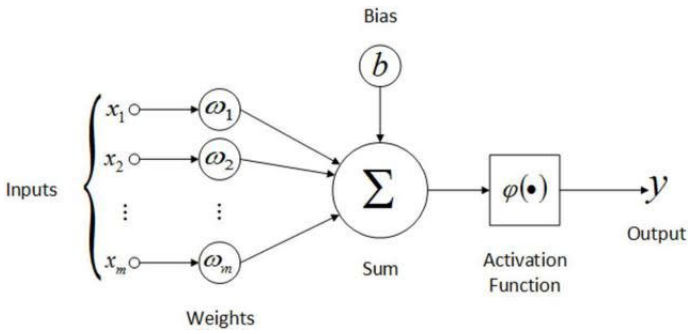


Figure 2.2: A single layer neural network

if we have an input vector $x \in \mathbb{R}^{m\times 1}$,a wights vector, $w \in \mathbb{R}^{m\times 1}$ then the output y of the perceptron is written as :

$$y = \varphi(W^T.x + b) = \varphi(\sum_{i=1}^{i=m} w_i x_i + b) \tag{2.2}$$

φ is an activation function, and it's useful to transform the linear form $W^T x$ to a non-linear from to be able to add layers after, otherwise without using an activation function a stack of layers can be reduced into a single layer.

for our single layer model, if we denote $z = W^T x + b$ then $y = \varphi(z)$
φ can be a sigmoid function :

$$f(x) = \frac{1}{1 + \exp\{-x\}} \tag{2.3}$$

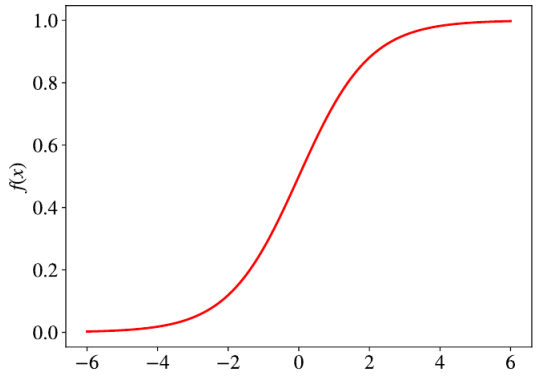


Figure 2.3: sigmoid function

and we have

$$y = \varphi(z) = \frac{1}{1 + \exp(-z)} \tag{2.4}$$

it could also be a **ReLU** (Rectified Linear Unit) function :

$$relu(x) = \begin{cases} 0 & x < 0 \\ x & otherwise \end{cases} \tag{2.5}$$

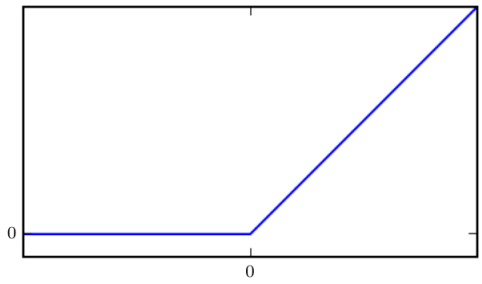


Figure 2.4: ReLU function

or a **tanh** function :

$$tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)} \tag{2.6}$$

if the single layer model is a multi-output model,$y \in \mathbb{R}^n$,then $W \in \mathbb{R}^{m \times n}$ in this case the activation function could be a **softmax**

$$f(x_i) = \frac{\exp(x_i)}{\sum_{i=1}^{n}} \tag{2.7}$$

which means the output of the perceptron is defined by

$$y_i = \varphi(w_i^T x + b_i) = \varphi(z_i) = \frac{\exp(z_i)}{\sum_{i=1}^{n} z_i} \tag{2.8}$$

where w_i is the i-th row of W, a softmax activation function produces probabilities that sum-up to 1

$$\sum_{i=1}^{n} y_i = 1 \tag{2.9}$$

and it is used for multi-class classification problems.

Now if instead of using a single layer we add more layers, then we will get a multi-layer neural network.

For a single output model with two layers, we will have a weights' matrix and a bias vector for each layer, $(W^{(1)}, b^{(1)})$ for the first layer and $(W^{(2)}, b^{(2)})$ for the second one.

if $x \in \mathbb{R}^{n\times 1}$ and $W^{(1)} \in \mathbb{R}^{n\times m}$ and $b \in \mathbb{R}^{m\times 1}$ then the output of the first layer will be :

$$y^{(1)} = \varphi^{(1)}(W^{(1)T}x + b^{(1)}) \tag{2.10}$$

$y^{(1)} \in \mathbb{R}^{m\times 1}$ will be the input for the second layer, which constraint the weights' matrix, $W^{(2)}$ to be a $\mathbb{R}^{(m\times 1)}$ since we have a single output $y \in \mathbb{R}$

$$y = \varphi^{(2)}(W^{(2)T}y^{(1)} + b^{(2)}) \tag{2.11}$$

if we chain two or more layers, we get a **deep** neural network, each layer transforms its inputs to a new representation and send it to the next one.

2.2.2 Supervised learning and optimization scheme

the work in this thesis is mainly focused on supervised problems where we have access to inputs and their targets.

In this context, learning means finding a set of values for the weights of all layers in the network, such that the network will correctly map example inputs to their associated targets.

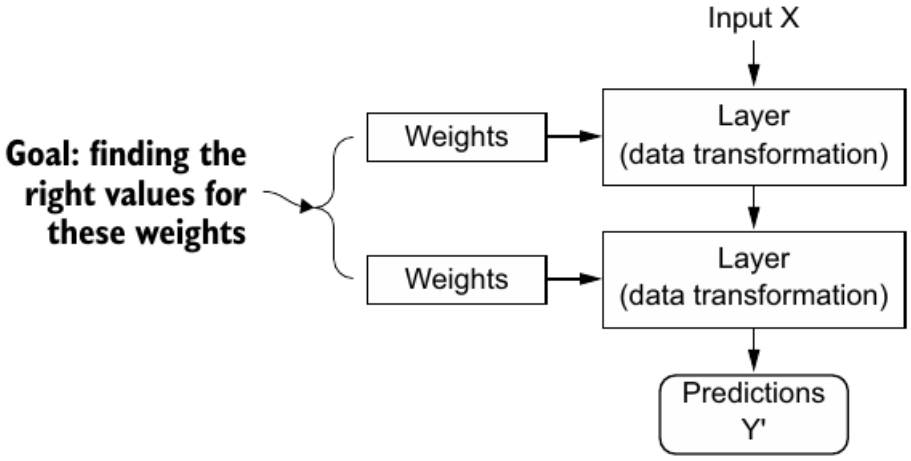


Figure 2.5: neural network (source [6])

To control the output of a neural network, we need to be able to measure how far this output is from what is expected. This is the job of the **loss** function of the network, also sometimes called the **objective function** or **cost function**.

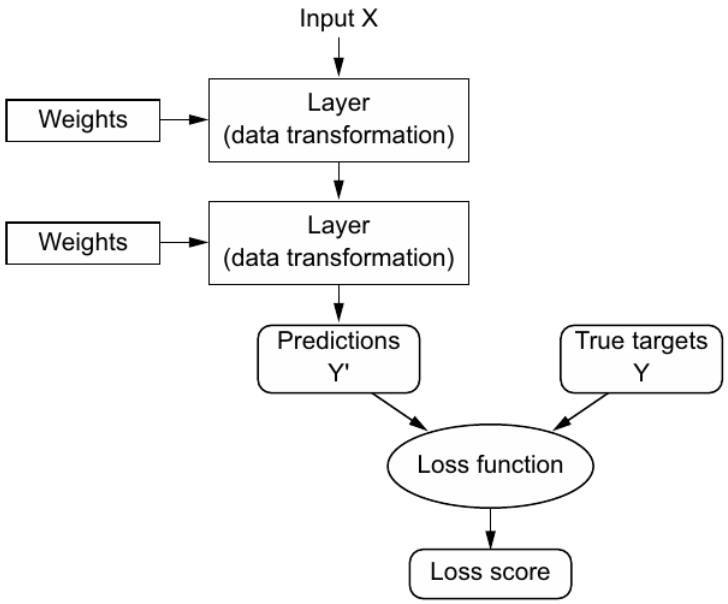


Figure 2.6: loss function [6]

The loss function takes the predictions of the network and the true target, and computes a distance score, capturing how well the network has done on this specific example figure 2.6.

The fundamental trick in deep learning is to use this score as a feedback signal to adjust the value of the weights a little, in a direction that will decrease the loss score for the current example.

This adjustment is the job of the **optimizer**, which implements what's called the **Back-propagation algorithm**: the central algorithm in deep learning.

2.2.3 Back propagation

To explain back propagation, we can take equation 2.10 which represents the output of a two layer neural network, first $(\mathbf{W}, \mathbf{b})$ need to be initialized (for example random initialization) if we take a vector input $\mathbf{x}$ and run the **forward pass** we will get a prediction $\hat{\mathbf{y}}$ that is far from the desired output at first.

We can for example compute the **Mean Squared Error(MSE)** defined by

$$MSE_{pred} = \frac{1}{m}\sum_{i}(\hat{y} - y)^2 = \frac{1}{m}\|\hat{y} - y\|_2^2 \tag{2.12}$$

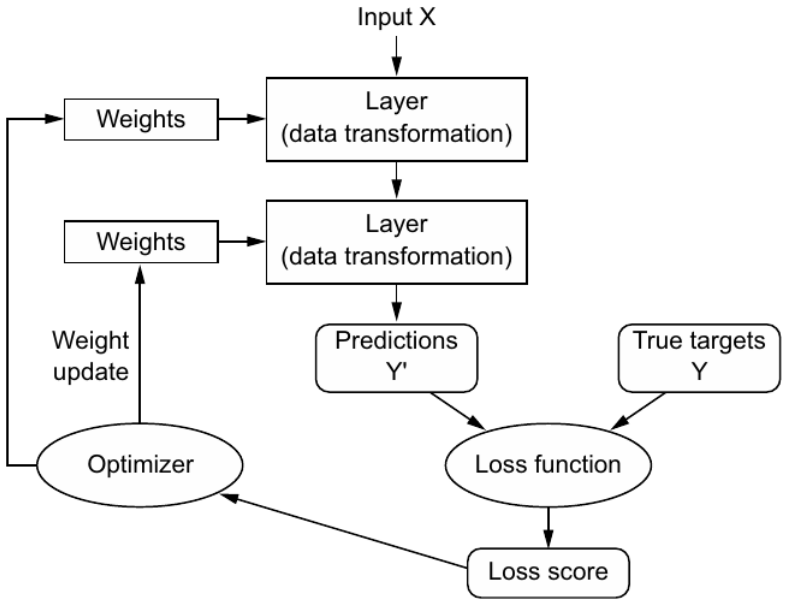


Figure 2.7: optimization [6]

so our loss function noted **J** is defined by

$$J(W^{(1)}, W^{(2)}, b^{(1)}, b^{(2)}) = \frac{1}{m}\left\|\varphi^{(2)}(W^{(2)T}y^{(1)} + b^{(2)}) - y\right\|_2^2$$
$$= \frac{1}{m}\left\|\varphi^{(2)}(W^{(2)T}(\varphi^{(1)}(W^{(1)T}x + b^{(1)})) + b^{(2)}) - y\right\|_2^2 \quad (2.13)$$

or using the sum where $\theta = W^{(1)}, W^{(2)}, b^{(1)}, b^{(2)}$

$$J(\theta) = \frac{1}{m}\sum_i (\varphi^{(2)}(W^{(2)T}(\varphi^{(1)}(W^{(1)T}x + b^{(1)})) + b^{(2)}) - y)^2 \quad (2.14)$$

the optimization process will update the set of variables $W^{(1)}, W^{(2)}, b^{(1)}, b^{(2)}$ by computing the gradient of J with respect to each one of them on the backward pass

$$W^{(1)} = W^{(1)} - \alpha\nabla_{W^{(1)}}J(\theta) \quad (2.15)$$
$$W^{(2)} = W^{(2)} - \alpha\nabla_{W^{(2)}}J(\theta) \quad (2.16)$$
$$b^{(1)} = b^{(1)} - \alpha\nabla_{b^{(1)}}J(\theta) \quad (2.17)$$
$$b^{(2)} = b^{(2)} - \alpha\nabla_{b^{(2)}}J(\theta) \quad (2.18)$$

for example, to compute $\nabla_{W^{(2)}} J(\theta)$ let's simplify by setting $(b^{(1)}, b^{(2)})$ to 0 and note

$$z^{(1)} = W^{(1)T} x \tag{2.19}$$

$$a^{(1)} = \varphi^{(1)}(z^{(1)}) \tag{2.20}$$

$$z^{(2)} = W^{(2)T} a^{(1)} \tag{2.21}$$

$$a^{(2)} = \varphi^{(1)}(z^{(2)}) \tag{2.22}$$

then we can write

$$J = \frac{1}{m} \left\| a^{(2)} - y \right\| \tag{2.23}$$

We can now apply the chain rule

$$\nabla_{W^{(2)}} J(\theta) = \frac{\partial J(\theta)}{\partial W^{(2)}} = \frac{\partial J(\theta)}{\partial a^{(2)}} . \frac{\partial a^{(2)}}{\partial z^{(2)}} . \frac{\partial z^{(2)}}{\partial W^{(2)}} \tag{2.24}$$

and we could do the same to compute $\nabla_{W^{(1)}} J(\theta)$

$$\nabla_{W^{(1)}} J(\theta) = \frac{\partial J(\theta)}{\partial W^{(1)}} = \frac{\partial J(\theta)}{\partial a^{(2)}} . \frac{\partial a^{(2)}}{\partial z^{(2)}} . \frac{\partial z^{(2)}}{\partial a^{(1)}} . \frac{\partial a^{(1)}}{\partial z^{(1)}} . \frac{\partial z^{(1)}}{\partial W^{(1)}} \tag{2.25}$$

α is a coefficient called a learning rate, and it determines how much we go in the direction opposite to the gradient, thus it modulates the speed of the gradient$(-)$.

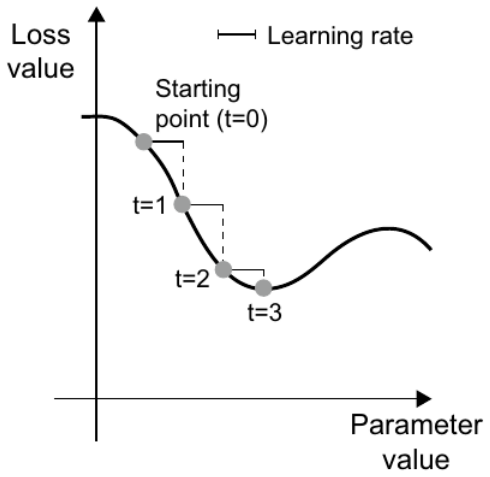


Figure 2.8: 1D loss curve [6]

updating the set of parameters following the set of equations 2.15... is what back propagation does, back propagating the gradients of the loss function to update the model parameters in such a way that minimizes the loss, doing the four steps, i.e :

- Run the forward pass

- compute loss
- compute gradients
- update the set of parameters

is called **gradient descent**

if we have a dataset of examples (signals, images ...) we can choose to run the four steps either by taking a single input example and its target at a time (a single sample such as a single image), make a prediction, compute loss, make the backward pass and update the parameters, this called **True Gradient Descent**, if the sample is taken randomly from the dataset then it's called **true Stochastic Gradient Descent SGD**.at first sight this seems inefficient.

We can instead of taking a single sample at a time take a **Batch of data** and do the four steps above, this is called **mini-batch SGD**

As an example, the dataset that I will use in this work (for the generation part) contains 4089 sample, if I take a batch size of size 128 then I will have $\frac{4089}{128} = 31$ which means I have 31 batches, the learning process will be as follows : take the first batch that contains 128 example (**Batch size**), run the 4 steps, then move to the second batch ..., once we do the 31 batches we have made a single iteration or **epoch**, now we can redo the same process again, and this is determined by the number of epochs.

The optimization process that is described by the set of equations (2.15) can be done by other algorithms such as **RMSprop** and **ADAM**

2.3 Convolutional and recurrent neural networks

2.3.1 Convolutional neural networks

Convolution neural networks (CNNs) were designed for image recognition tasks and were originally applied to the challenge of handwritten digit recognition [10].

The basic design goal of CNNs was to create a network where the neurons in the early layer of the network would extract local visual features (such as lines at a particular angle, or segments of curves), and neu- rons in later layers would combine these features to form higher-order features (such as eyes or noises).

As opposed to densely connected layers (like the two layers neural network discussed previously) CNNs learn local patterns whereas that dense layers learn global patterns this describes CNNs with another interesting propriety [6]

- **CNNs learn patterns that are translational invariant**: After learning a certain pattern in the lower-right corner of a picture or a pattern on the left side of a time series, a CNN can recognize it anywhere.

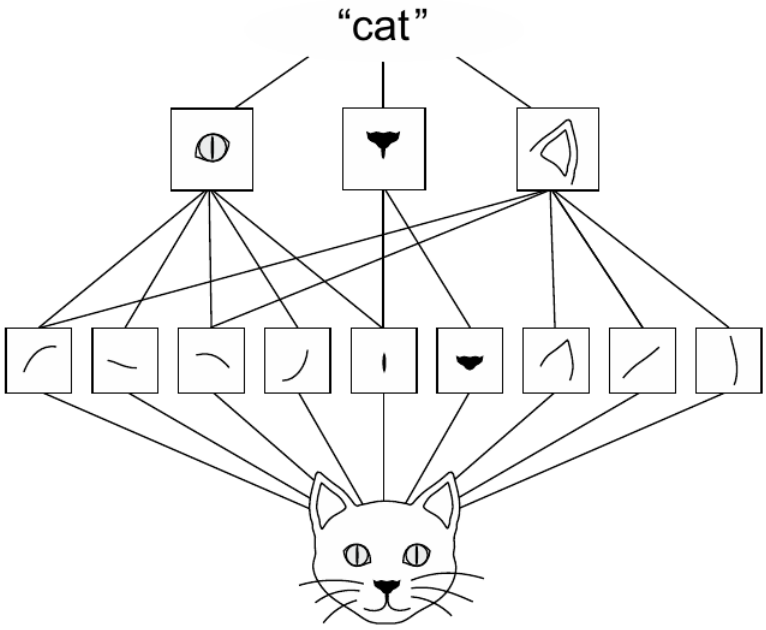


Figure 2.9: the decomposition of a cat face into a hirarchy of visual modules [6]

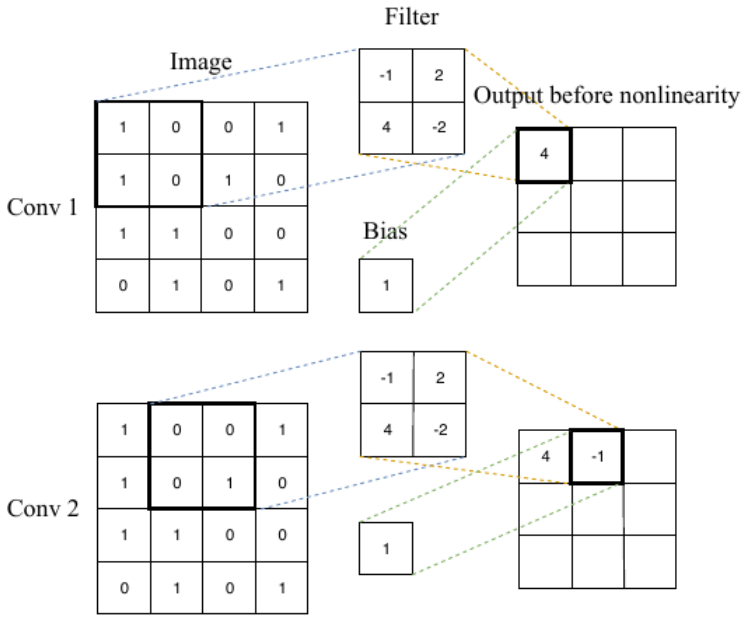


Figure 2.10: Convolution operation over an image [7]

Convolution can be performed by moving a window (called a **filter**) over the input as depicted in fig 2.10

One layer of a CNN consists of multiple convolution filters, and during training, a CNN

layer learns those filters.

if we want to get an output with the same size as the input after a convolutional operation, we need to use **padding** (adding 0's to the border of the input) and if we want to downsample the size of the output, we use **strides** (steps to move the filter over the input).

Padding and strides are central to build the Encoder and Decoder of an autoencoder when using CNNs.

2.3.2 Recurrent neural networks

A major characteristic of densely connected networks and convnets, is that they have no memory. on the other hand, recurrent neural network **(RNN)** processes sequences by iterating through the sequence elements and by maintaining a **state** that contains information relative to what it has seen so far.This is very important when sequence order matters, for example if we want to use deep learning on time series, the natural choice would be to select neural networks that take into account order information (or time).

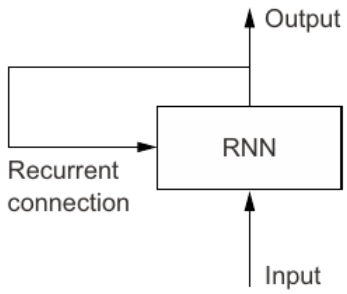


Figure 2.11: a recurrent network [6]

Since an RNN iterates through sequence elements, it has an internal loop, the state of the RNN is reset between processing two different, independent sequences (such as two samples in batch), so actually the network still consider one sequence to be a single data point (a single input to the network). What changes is that this data point is no longer processed in a single step, rather, the network internally loops over its elements.

the output of an RNN at time-step t is given by

$$output_t = \varphi(W^T.input_t + U^T.state_t + b) \tag{2.26}$$

where W and U weight tensors that will be updated during training fig 2.12 shows a simple RNN

In practice, using a simple RNN like the above architecture is not of real use, if the sequences are too long then a simple RNN would suffer vanishing gradients (the gradients are overwhelmed by noise and back propagation won't work), it's the same problem faced by feed forward networks like densely connected layers and CNNs ,if we keep adding layers and go deep, the network becomes untrainable ,for CNNs and densely connected layers the

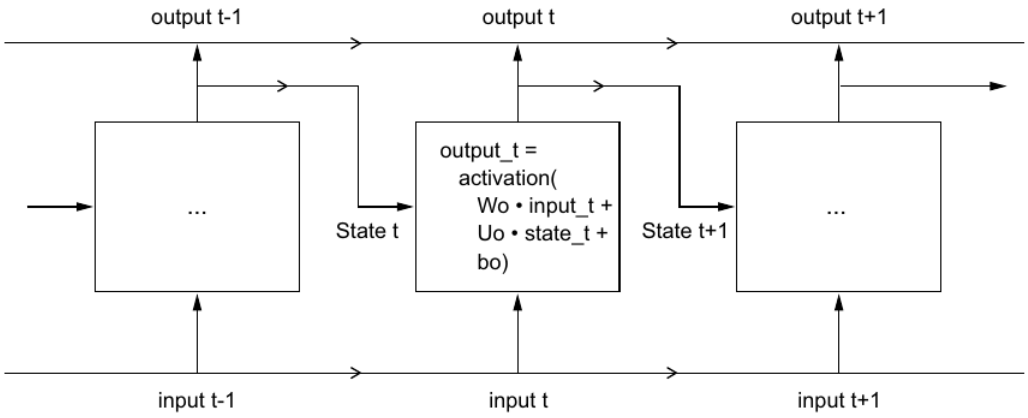


Figure 2.12: a simple RNN with 3 units [6]

solution was to use **Residual connections** which is adding a layer input to its output, so the next layer doesn't lose information about past inputs [6].

to address the issue of vanishing gradients, **Long Short-Term Memory(LSTM**) and **Gated Recurrent Units(GRU)** were designed.

2.4 Deep learning for data generation

The key idea of data generation is to develop a low-dimensional space, (**latent space**) of representations, where any point can be mapped to a **"valid"** data: a data that looks like the real data in other words it acts like compression where we have data that have a specific form we compress it tow lower representation and then decompress it. The module capable of realizing this mapping, taking as input a latent point and outputting an (image or a time series), is called a generator (in the case of Generative adversarial networks) or a decoder (in the case of auto-encoders). Once such a latent space has been learned, we can sample points from it, and, by mapping them back to the data space, we can generate never seen data that are in between training data.[6]

2.4.1 Auto-Encoders

A classical autoencoder takes an input, maps it to a latent vector space via an encoder module, and then decodes it back to an output with the same dimensions as the original input, via a decoder module.

It's then trained by using as target data the same inputs, meaning the autoencoder learns to reconstruct the original inputs. By imposing various constraints on the code (the output of the encoder), like making the dimension of the latent space a low-dimensional and sparse,

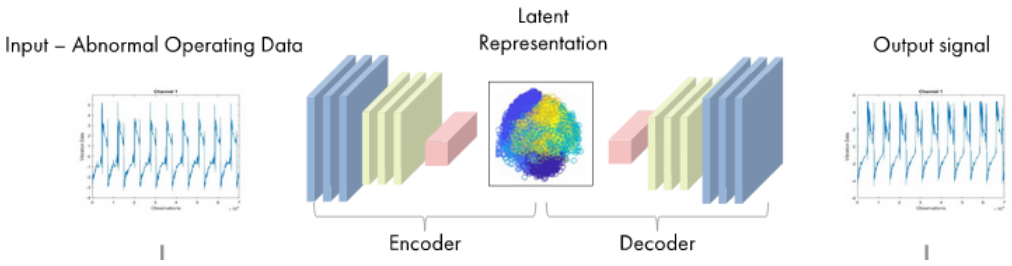


Figure 2.13: auto-encoder

we can get the autoencoder to learn more- or less-interesting latent representations of the data.

In such case, each input will have a fixed code in the latent space, which forces the latent space to be sparse, this make difficulties when we sample from the latent space.

2.4.2 Variational auto encoders VAEs

Variational autoencoders **VAEs**, augment autoencoders with a little bit of a what can be interpreted as a statistical magic, and what forces VAEs to learn continuous structured latent spaces as opposed to auto-encoders [6].

A VAE, instead of compressing its input into a fixed code in the latent space (auto-encoder), turns the input into the parameters of a statistical distribution (simply two vectors of numbers): a mean and a variance, this means we're assuming the input data has been generated by a statistical process, and that the randomness of this process should be taken into account during encoding and decoding.

The VAE then uses the mean and variance parameters to randomly sample one element of the distribution i.e, the sampling process is simply taking the two vectors, the compression result by the encoder or the code and add to them a random vector that follows the normal distribution what will this cause is changing the position of that code in the latent space to a different location each time the same input is provided to the encoder, during the training process), the same input code will have different location in the latent space, the decoder will decode that element back to the original input from different codes ((this what forces the latent space to be continuous).

The stochasticity of this process forces the latent space to encode meaningful representations everywhere: "every point sampled in the latent space is decoded to a valid output.[6]".

in simplest, terms, for each input the encoder codes this input to two vectors in the latent space,z_{mean} and $zlog_{variance}$,from those two vectors we sample a point z according to the following equation

$$z = z_{mean} + \exp(zlog_{variance}) * \epsilon \tag{2.27}$$

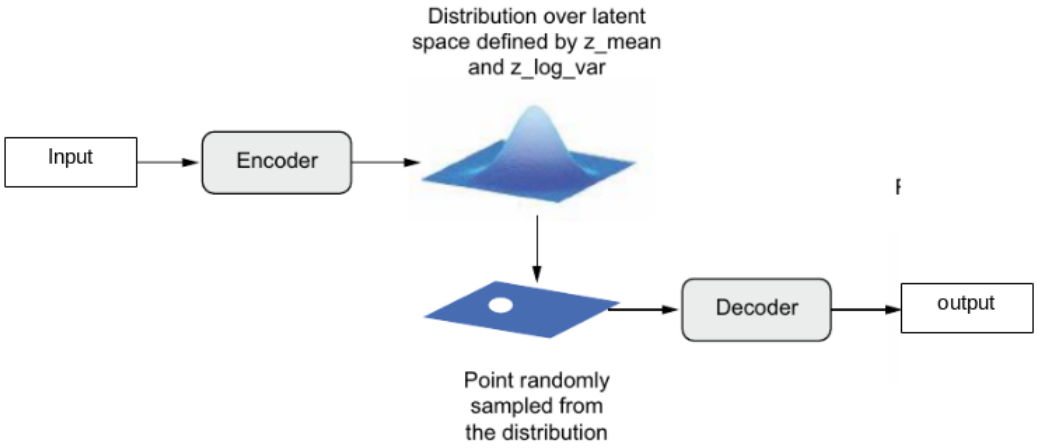


Figure 2.14: variational auto-encoder

where ϵ is a random tensor of small values.

Because epsilon is random, the process ensures that every point that's close to the latent location where we encoded an input (z-mean) can be decoded to something similar to the same input, thus forcing the latent space to be continuously meaningful, However if the sampled point is very far from all coded inputs the code of an original input it will decode just noise.

2.4.3 Generative adversarial networks(GANs)

Generative adversarial networks (GANs), introduced in 2014 [11] are an alternative to VAEs for learning latent spaces of data.

A GAN consists of two deep neural networks exactly like a VAE,

- A **generator network** :Takes as input a random vector (a random point in the latent space), and decodes it into a synthetic data
- **Discriminator** network (or adversary), takes an input (real or synthetic), and predicts whether this input came from the training set or was created by the generator network

if we note by G the generator networks, by D the Discriminator network, by $z \sim P_z$ a random vector sampled from the latent space (random noise) that is drawn from a prior distribution (we use Normal distribution in this work), and by $x \sim P_{data}$ an input.We train D to maximize the probability of assigning the correct label to both training examples and

samples from G (like assigning 1 to training data and assigning 0 to generated data). We simultaneously train G to minimize $\log(1 - D(G(z)))$[11]:

this means the discriminator D and the Generator G plays a min-max game with a value function V:

$$min_G max_D V(D, G) = \mathbb{E}_{x \sim p_{data}}[log(D(x))] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))] \tag{2.28}$$

The generator network is trained to be able to fool the discriminator network, and thus it evolves toward generating increasingly realistic data as training goes on.

the optimization of D and G are performed according to the following equations :

updating the discriminator by **ascending** its stochastic gradients

$$\nabla_{\theta_D} \frac{1}{m} \sum_i [\log(D(x_i)) + \log(1 - D(G(z_i)))] \tag{2.29}$$

updating the generator by descending its stochastic gradient

$$\nabla_{\theta_G} \frac{1}{m} \sum_i [\log(1 - D(G(z_i)))] \tag{2.30}$$

where θ_D, θ_G are the parameters of both the discriminator and generator.[11]

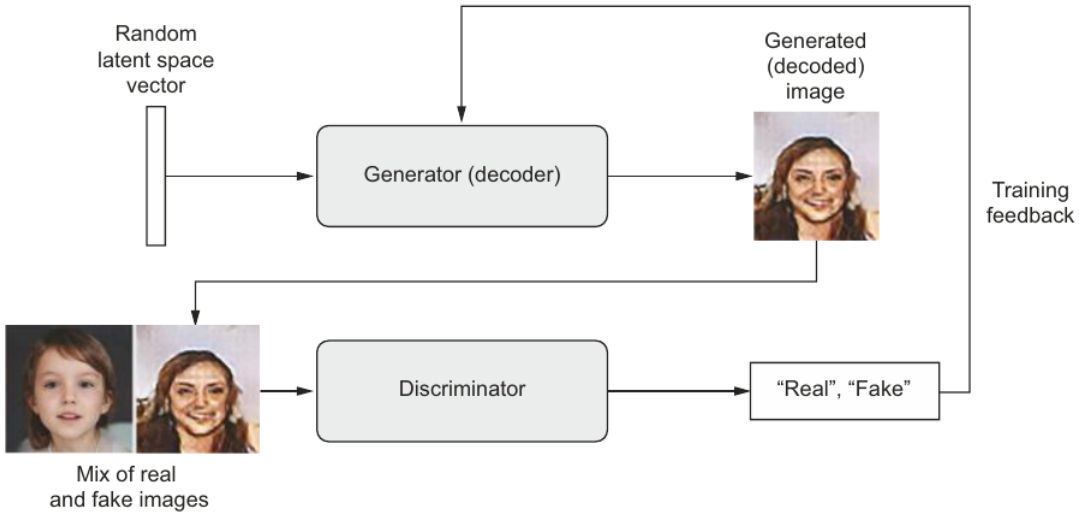


Figure 2.15: the generator is trained to fool the discrimiator [6]

2.4.4 Maximum mean discrepancy MMD

To be able to measure how close the generated data to the real data, we can assume that the train data was generated by a random process which means if we note T the train set

$\forall x \in T\ x \sim P$ also the model either a VAE or a GAN will generate data and if we note G the set of generated data then $\forall y \in G\ y \sim Q$ where P, Q are two probability distributions, the goal of training is to make Q close to P.

We test whether distributions P and Q are different on the basis of samples drawn from each one of them, by finding a smooth function which is large on the points drawn from P, and small (as negative as possible) on the points from Q. We use as test statistic, the difference between the mean function values on the two samples; when this is large, the samples are likely from different distributions. We call this test statistic the **Maximum Mean Discrepancy (MMD)** [12].

if $\mathcal{F}$ is a class of functions, $f : \mathfrak{X} \to \mathbb{R}$ and $x \in T$ is a random variable,$y \in G$ is a random variable, $X = \{x_1, x_2, x_3..., x_m\}$ is a set of observations of x,$Y = \{y_1, y_2, y_3, ..., y_n\}$ is a set of observations of y then MMD is defined as:

$$MMD[F, P, Q] = sup_{f \in \mathcal{F}}[\mathbb{E}_{x \sim P}(f(x)) - \mathbb{E}_{y \sim Q}(f(y))] \tag{2.31}$$

Since we don't know a prior the probability distributions P and Q we need to use biased or unbiased estimators to compute $\mathbb{E}(f(x))$ and to be able to implement MMD numerically.

a **biased empirical estimate of the MMD** is

$$MMD_{biased} = sup_{f \in F}(\frac{1}{m}\sum_{i=1}^{m} f(x_i) - \frac{1}{n}\sum_{j=1}^{n} f(y_j)) \tag{2.32}$$

in this work we interested in MMD in a **reproducing kernel Hilbert space RKHS** since by definition an RKHS is a Hilbert space of functions where if f and g are two functions close in norm, i.e. $\|f - g\|$ is small then f and g are also pointwise close, i.e. $|f(x) - g(x)|$ is also small $\forall x \in \mathfrak{X}$.

if $\mathcal{H}$ is an RKHS of functions, there is a feature mapping $\varphi(x)$ from $\mathfrak{X}$ to $\mathbb{R}$ such that :

$$f(x) =< f, \varphi(x) >_{\mathcal{H}} \tag{2.33}$$

This feature mapping has a **canonical** form $\varphi(x) = k(x, .)$ and it's called a **kernel**

[12] defined also $\mu_p \in \mathcal{H}$ such that $\mathbb{E}_{x \sim P}(f) =< f, \mu_p >_{\mathcal{H}}\ \forall f \in \mathcal{H}$ and it's called a **Mean Embedding** of the probability distribution P.

Now if we set f to, $f = \varphi(t) = k(t, .)$ we get, $\mu_p(t) =< \mu_p, k(t, .) >_{\mathcal{H}} = \mathbb{E}_{x \sim P}(k(t, x))$ which means the mean embedding of the probability distribution P is the expectation of the canonical feature map $k(x, .)$.

With this we will have now

$$MMD^2(\mathcal{F}, P, Q) = \|\mu_P - \mu_Q\|^2_{\mathcal{H}} =< \mu_P, \mu_P >_{\mathbb{R}} + < \mu_Q, \mu_Q >_{\mathbb{R}} -2 < \mu_P, \mu_Q >_{\mathbb{R}} \tag{2.34}$$

If we have x_1, x_2 tow independent random variables from the train set T ,y_1, y_2 two (assumed) independent random variables from the generated set G then equation 2,34 is written as

$$MMD^2(\mathcal{F}, P, Q) = \mathbb{E}_{x_1 \sim P, x_2 \sim P}(k(x_1, x_2)) + \mathbb{E}_{y_1 \sim Q, y_2 \sim Q}(k(y_1, y_2)) - 2\mathbb{E}_{x_1 \sim P, y_1 \sim Q}(k(x_1, x_1)) \tag{2.35}$$

In my work I used the empirical unbiased estimator of equation 2.35 which is

$$MMD^2(\mathcal{F}, X, Y) = \frac{1}{m(m-1)} \sum_{i=1}^{m} \sum_{j=1, i \neq j}^{m} k(x_i, x_j) + \frac{1}{n(n-1)} \sum_{i=1}^{n} \sum_{j=1, i \neq j}^{n} k(y_i, y_j) - \frac{2}{mn} \sum_{i=1}^{m} \sum_{j=1}^{n} k(x_i, y_j) \tag{2.36}$$

the kernel function chosen is the Gaussian kernel

$$k(x, y) = \exp\left(-\frac{(x-y)^2}{4}\right) \tag{2.37}$$

2.5 The Transformer architecture

The Transformer architecture, developed in 2017,started overtaking most natural language processing tasks, and after other tasks like music generation and [8], this architecture is at the core of generative pretrained transformers(GPT).

The transformer have an **Encoder-Decoder** structure, and leverage **neural attention**, which is a simple mechanism yet powerful that enabled building powerful sequence models that didn't feature any recurrent layers or convolutional layers.

Neural attention can be explained by making an analogy of a person reading a book, he will be focusing on what is most important to him, skipping what is not of much importance, like giving a score to each word and extracting what is important, adding neural attention to a deep learning model will enable it to apply attention to what is relevant to the task that need to be solved.

In a transformer, the encoder maps a sequence of symbol representation $x = \{x_1, x_2, ..., x_n\}$ to a new sequence of representation $z = \{z_1, z_2, ..., z_n\}$ given z the decoder then outputs a new sequence $y = \{y_1, y_2, ..., y_m\}$

Although, a transformer could be used to generate time-series like ECG's, in this work I will be only interested in using the encoder part only for detecting R-peaks so I need to describe the attention mechanism and positional embedding.

2.5.1 The attention mechanism (self attention)

When reading about the transformer we often encounter a set of terms, Query, key and values because the transformer was originally designed for language translation, it turns a sequence to another sequence, for the case of using only the transformer encoder the (query, key and value) mean the same input which is a time sequence $\mathbf{x} = \{\mathbf{x_1}, \mathbf{x_2}, ..., \mathbf{x_n}\}$,yet the

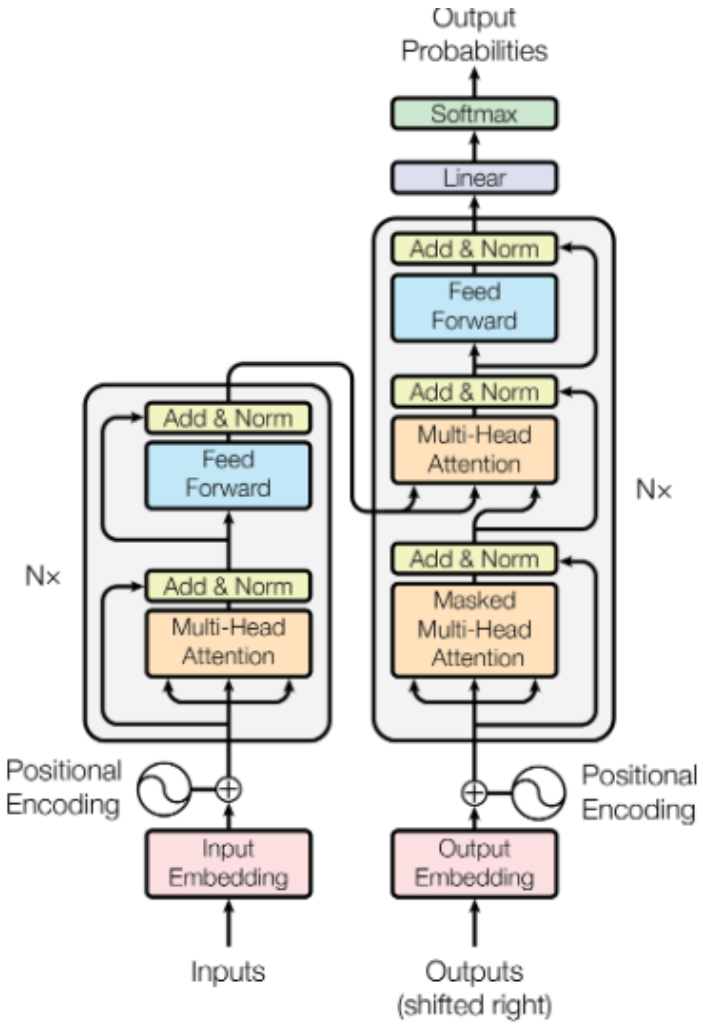


Figure 2.16: the transformer architecture [8]

set of equations that will describe the operation used to compute attention scores contains letters that refer to those 3 terms. for the input sequence x mentioned above, the output of the attention is a new sequence $\mathbf{z} = \{\mathbf{z_1}, \mathbf{z_2}, \mathbf{z_3}, ..., \mathbf{z_n}\}, z_i \in \mathbb{R}^{d_z}$,d_z is the embedding space dimension that I'll describe shortly in the positional encoding section.

z_i is computed according to

$$z_i = \sum_{i=1}^{n} \alpha_{i,j}(x_j W^V) \tag{2.38}$$

and each coefficient weight $\alpha_{i,j}$ is calculated using a softmax function :

$$\alpha_{i,j} = \frac{\exp\{e_{ij}\}}{\sum_{k=1}^{n} \exp\{e_{ik}\}} \tag{2.39}$$

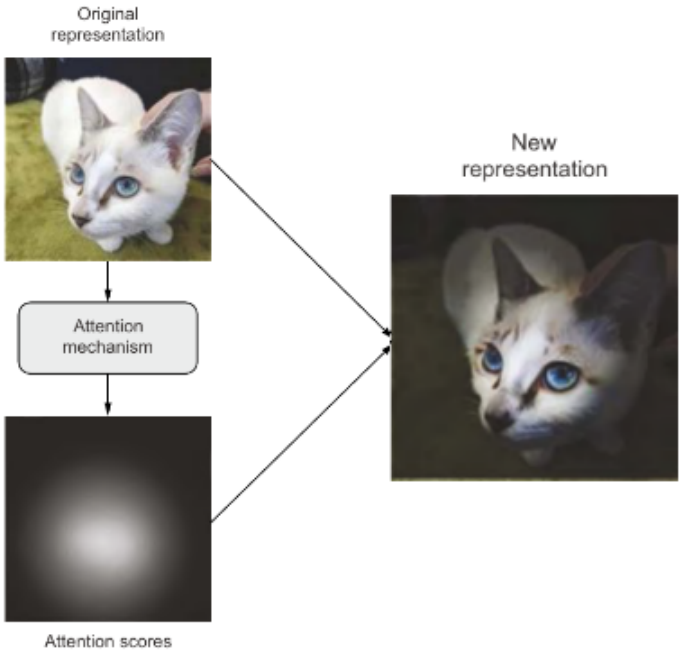


Figure 2.17: attention [6]

e_{ij} is an attention wight or score and is computed using a scaled dot product :

$$e_{ij} = \frac{(x_i W^Q)(x_j W^K)^T}{\sqrt{d_z}} \tag{2.40}$$

W is a weight matrix that is learned during training.

equation 2.40 represents a dot product between two vectors $x_i W$ and $x_j W$ a dot product is a measure of distance, so e_{ij} returns simply the distance between x_i and x_j and equation2.39 calculates weights of relevancy between each element of the input sequence and the rest of elements, and equation 2.38 is a weighted sum, if an element x_i, is not relevant to the element x_j the weight α_{ij} will be close to 0, so element x_j will compute almost nothing to the weighted sum, on the other hand if an element x_k is closely related to x_j then it will contribute more to the sum, this results on with w new vector representation for element x_i, Fig 2.18

Multi-Head-attention used in the transformer is self attention computed multiple times in parallel and the result is a concatenated.

2.5.2 Positional Encoding

Since the transformer doesn't contain any recurrent neural networks, it can't preserve the order of a time series, so the information of time or position should be added, in this work we have used the absolute positional encoding that was proposed in the attention all you need

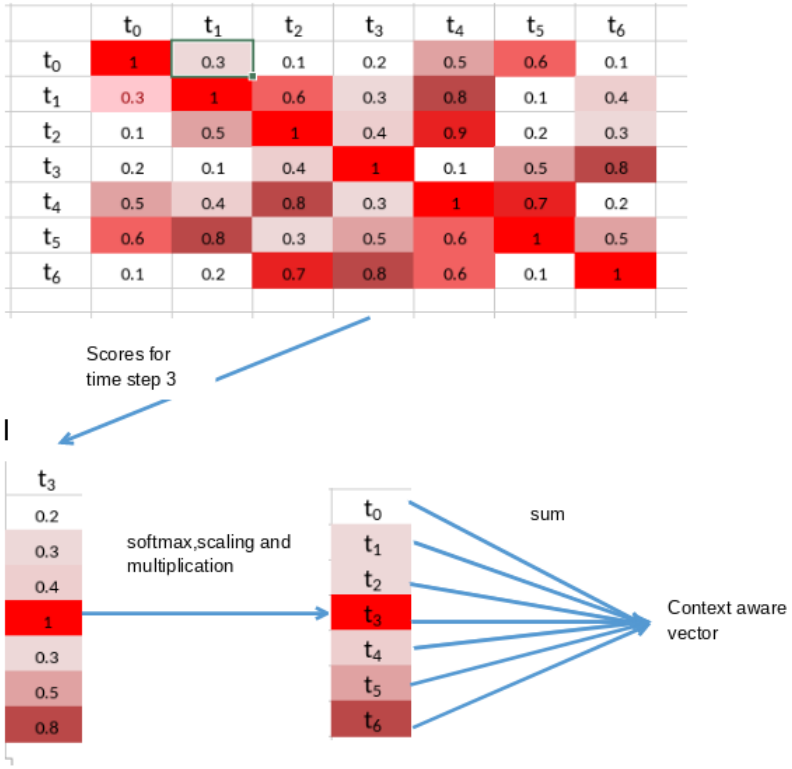


Figure 2.18: self attention

paper [8] was used, this encoding is based on adding a position vector to the input, if we have a position vector $P\{= p_1, ..., p_n\}$ then for each element input we add to it its position:

$$x_i = x_i + p_i \tag{2.41}$$

$p_i \in \mathbb{R}^{d_z}$ first, to be able to add such quantities the input must have the same dimension as the position vector, for example if x_i is the value at time step t_i it should be projected into a higher dimensional space, this is basically called embedding, such an embedding for time series could be achieved either by using convolutional layers, or simply by a simple matrix multiplication (dense projection),

$$x = x.W + p \tag{2.42}$$

for a time series, position are integers that can be too large, like 512 time steps signal, a neural network can ingest large numbers, the trick was to use sin and cosine functions with that p_i is computed according to :

$$p_i(2k) = sin(iw_k) \tag{2.43}$$

$$p_i(2k+1) = cos(iw_k) \tag{2.44}$$

$$w_k = 10000^{\frac{-2k}{d_z}} \tag{2.45}$$

and k is in the range $[0, \frac{d_z}{2}[8]$

Variations to w_k ensure that no positions $< 10^4$ are assigned similar embeddings.

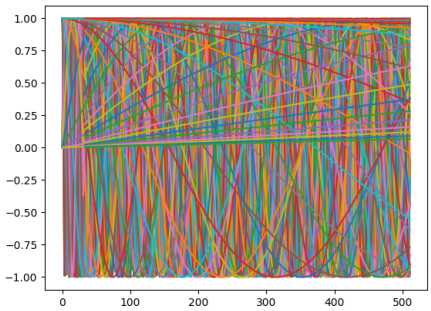


Figure 2.19: a position encoding vector of a 512 ECG signal

2.5.3 Transformer Encoder

The transformer encoder is the first part of the transformer architecture, and it consists of a stack of basic neural operations, we first embed the input into a higher dimensional space, this can be achieved with a dense projection or a convolutional operation, then we add positional encoding and then the input will be processed in the multi-head attention block, the encoder block has dense projections and residual connections, so information can't be lost for a sufficient deep network this block is depicted in the next figure, it's the part used in the transformer encoder based U-net model.

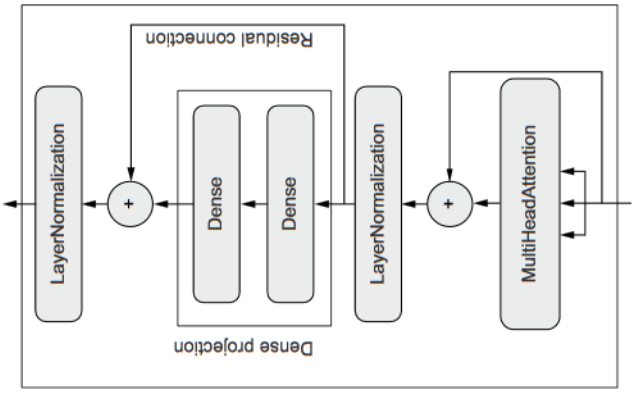


Figure 2.20: transformer encoder

2.6 R-Peak detection as a segmentation problem

To be able to train a model to detect R-peaks, we treat the problem as a segmentation problem where we will train a U-net like model, that is an auto-encoder like model, to segment a time series by generating binary masks, and then we extract exact R-peak positions from those masks.

2.6.1 Types of segmentation

Segmentation was basically applied for images (computer vision tasks), and it consists of partitioning an image to different areas, with each area usually representing a category. Segmenting time series or images consists of training a model that assigns a class to each pixel in an image and for time series to each time-step (some sort of multi class classification problem). There are two types of segmentation for images, :

- **Semantic segmentation** where each pixel is independently classified into a semantic category, like "cat." If there are two cats in the image, the corresponding pixels are all mapped to the same generic "cat" category (for example, we assign a pixel value 1 for all cats whatever their type is). this type of segmentation will be used for the task of R-peak detection, where we segment a signal to QRS-complex as a first category and the rest as a second category.
- **Instance segmentation,** which seeks not only to classify image pixels by category, but also to parse out individual object instances. In an image with two cats in it, instance segmentation would treat "cat 1" and "cat 2" as two separate classes of pixels.

A used mask for QRS segmentation is depicted in the below figure, it's a binary mask where time steps that belong to a QRS-complex are assigned the value 1 and other time steps are assigned 0.

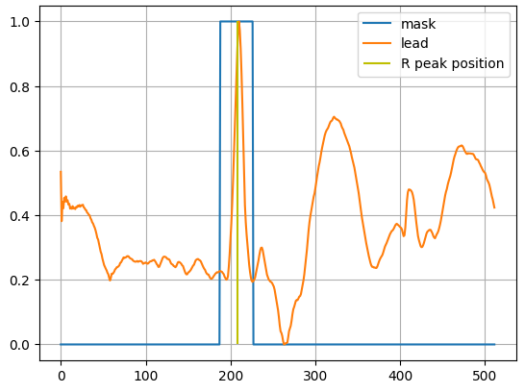


Figure 2.21: an example of a mask

2.6.2 U-net architecture

The U-net architecture consists of a CNN encoder and a CNN decoder, added to this is a copy and crop operation that transfers data from the encoder to the decoder part, the cropped area from the feature map of the encoder part is concatenated to the feature map produced by the convolutional transpose part, the operations used are described by the arrows in figure 3.22,to simplify further the idea behind this network, instead of using residual connections (described in the previous chapter) to solve vanishing gradient problem, U-net designer used some sort of skip connections instead.

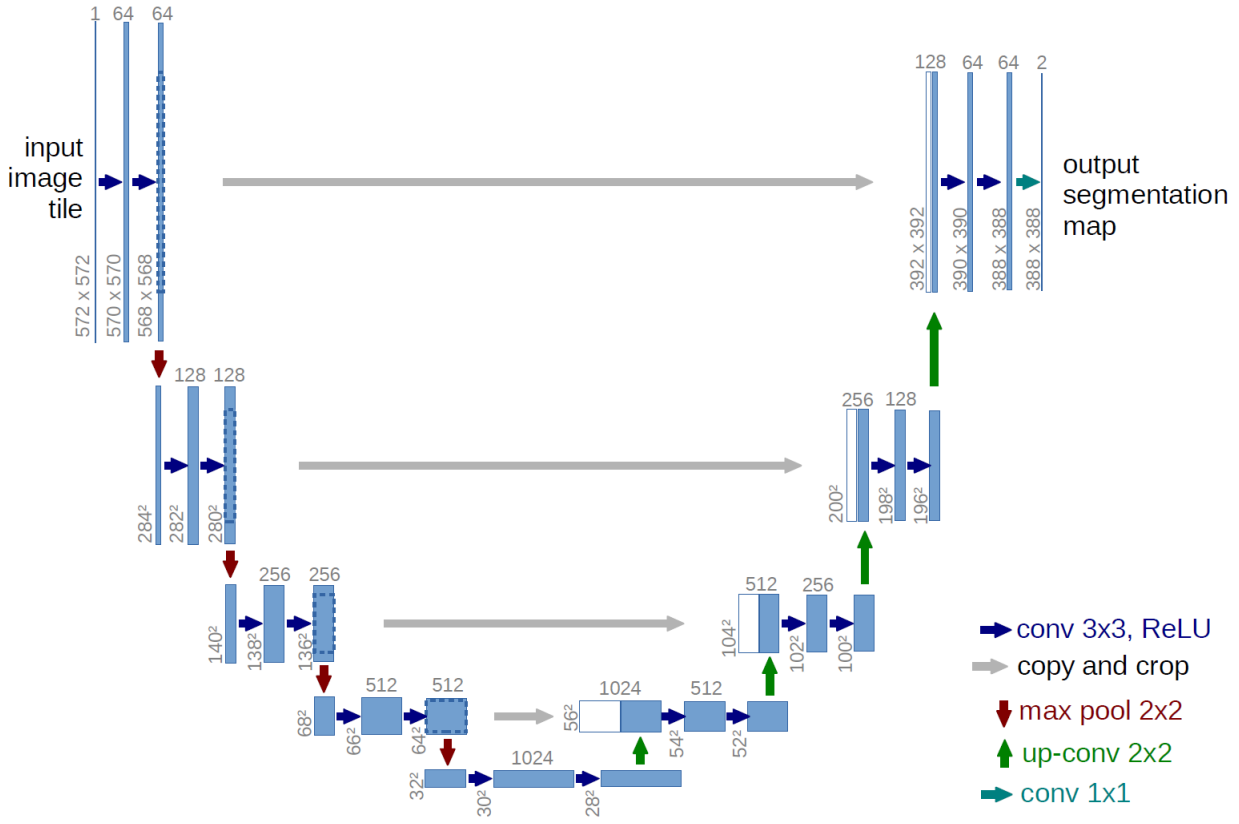


Figure 2.22: Unet Architecture [9]

Epsidy uses a modified network of U-net for signal segmentation that I will compare its performances with a U-net like min-Xception network and the transformer encoder based network that I developed.

2.6.3 Mini-Xception U-Net like

This network is a modified version of the original Xception network developed by Francois Chollet that is based on inception with the difference of introducing the use of separable

convolutions. in this work, a modified and a lightweight model has been used for R-peak detection [13]. The network graph is attached in Appendix A

2.6.4 U-net like transformer-encoder based network

another U-net like network that feature attention with the integration of a Transformer encoder will be used for segmentation, the network graph is shown in f. The network graph is attached in appendix B.

2.6.5 Model validation

To monitor if the segmentation model is giving good performances or not, we used Leave One Out Cross-validation(**LOOCV**) which is a variant of K-fold cross validation, which means out of the 6 recordings, we use 5 for training and the remaining for validation. The metrics that are computed for a segmentation problem are :

- **True Positives(TPs)**: how many detected R-peaks that are true R-peaks
- **True Negatives(FNs)** : how many missed R-peaks that are actually R-peaks.
- **False Positives FPs** : how many R-peaks that the model saw as R-peaks, but they are not.

we also use **Recall, Precision and F1-score** to assess the model performance

$$precision = \frac{TPs}{TPs + FPs} \tag{2.46}$$

$$recall = \frac{TPs}{FNs + TPs} \tag{2.47}$$

$$F1Score = \frac{2 * recall * precession}{recall + precession} \tag{2.48}$$

It's important to note that, if a model produces a lot of FPs the triggering of RF pulses will produce low quality MR images and a model that produces a lot FNs will cause a skipping of R-peaks and hence no RF-pulse triggering which will cause an increase of MR time, so there must be a balance between the two, but most importantly if FP's is small the model is good, this can be measured by precision, the higher the precision value is, better the model is.

2.7 Conclusion

In this chapter, I described briefly what is deep learning and how deep models learns, described the CNNs and LSTMs that were used in my work, VAEs and GANs, and the transformer encoder that I used to build an R-peak detector.

Chapter 3

Realistic Data generation and R peak detection

3.1 Introduction

In this chapter I will be introducing the dataset that has been used to train both generative and detection models, the architecture graphs of the used models, the results of the generation and R-peak detection, and a comparison between three detection models.

3.2 Dataset description and dataset scaling

The dataset that will be used to train generative, and detection models is a part of the Krug's dataset publically available at physionet (Krug's dataset) [14].

The dataset comprises 43 records from 23 different subjects with an overall length of 203 min.
The original data files are provided in the WFDB format and the sampling rate was 1024 Hz. In this work we focused only on using the record of 6 subjects, that are 12 leads recordings, under a field strength of 3 Tesla, and the measurement position inside MR is Head First(HF), the data files are resampled to 500Hz.

To be able to train generative models without losing the information about where R-peaks are, we used a fixed R position slices, to do so we locate an R-peak position and extract a window constructed by 25% of the total length before and by 75% of total length after.

Also, before training a deep learning model, the data should be normalized, so the network can ingest, an example of normalization is to scale the data by subtracting the mean and dividing by the standard deviation, for the data generation part a scaling between [0,1] or [-1,1] of the training data is sufficient and gives better results in terms of the versatility of the generated data.

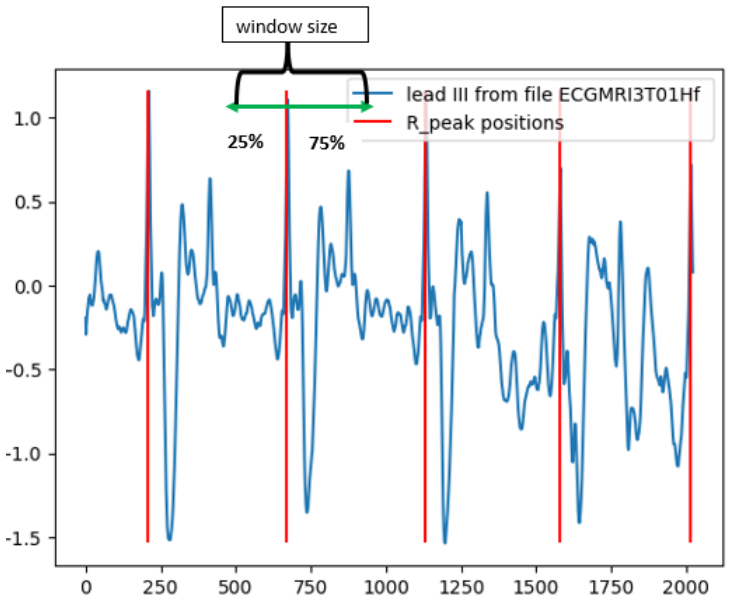


Figure 3.1: slicing data

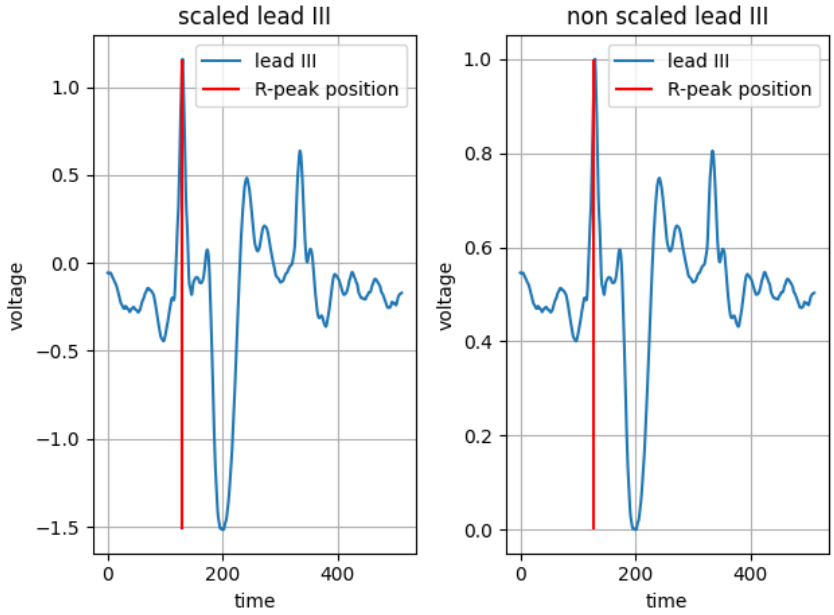


Figure 3.2: scaling data

The below figure shows the 12 leads in 512 time steps slices with fixed R-peak position. we can see from the figure 3.3 that the R-peak can't be discriminated by a human eye,

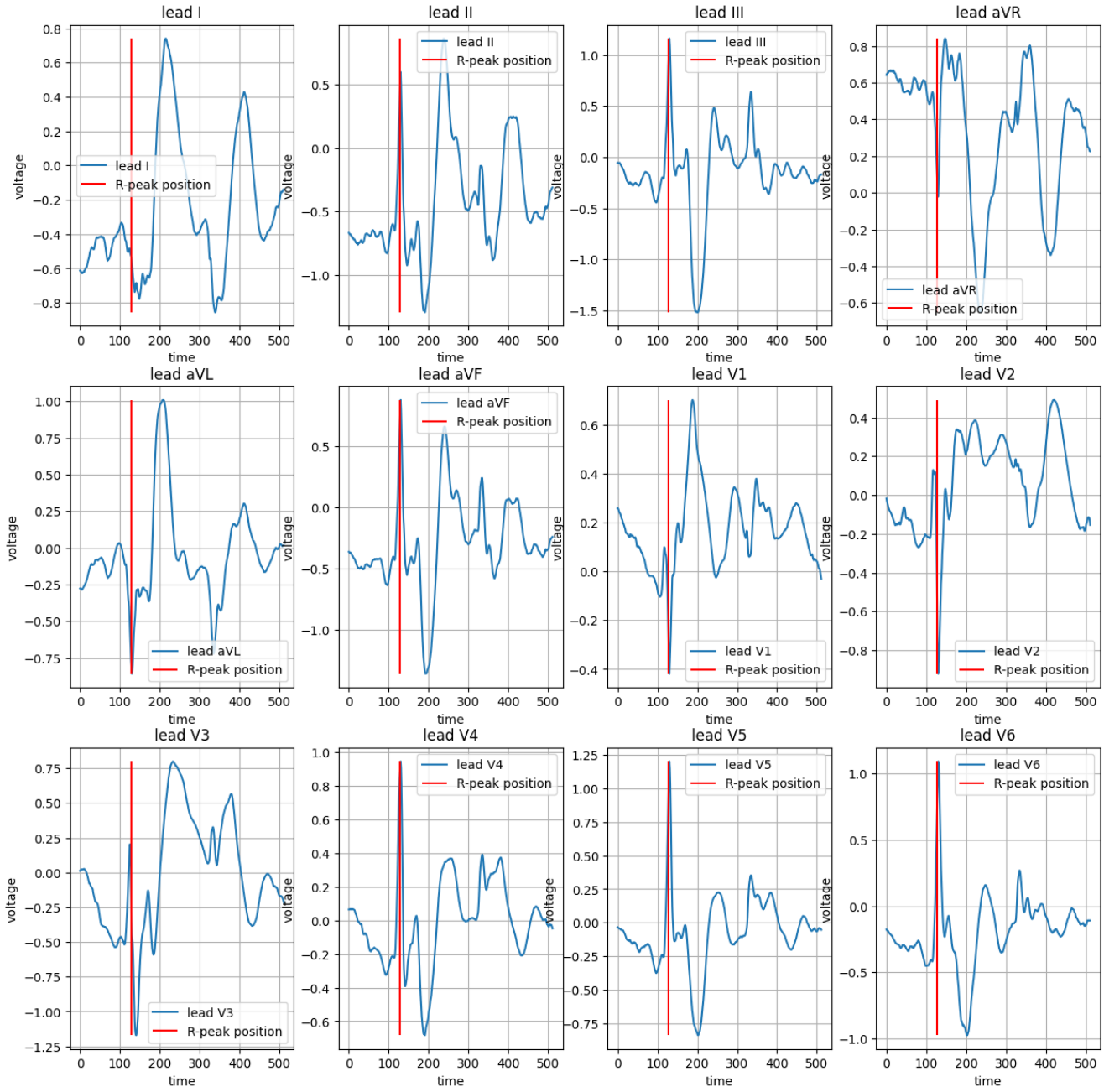


Figure 3.3: 12 leads 512-timesteps slices

and we can know that there is an R-peak on that position based on the waveforms of other leads, or by seeing that the high amplitude spark on lead I is the T-wave amplified by the MHD.

3.3 Generative models

The next diagram summarizes the models that will be used for generation. A CNN-CNN VAE (respectively a GAN) is A VAE (respectively a GAN) where the encoder is a stack of convolutional neural networks layers and the decoder as well,

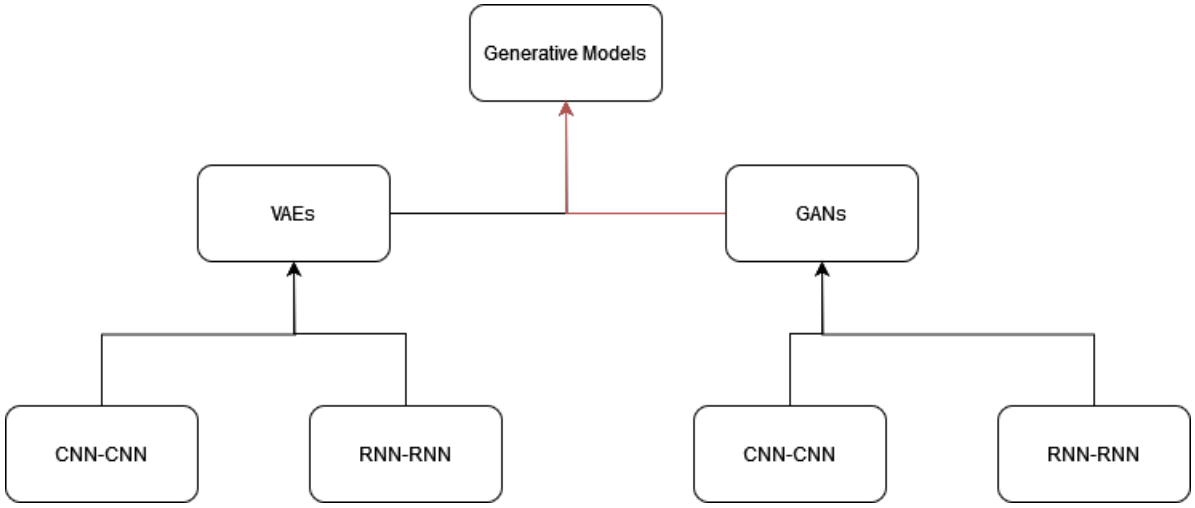


Figure 3.4: Generative models

3.3.1 CNN-CNN and LSTM-LSTM VAEs

For a CNN-CNN vae the encoder and decoder model graphs are depicted in the following figures.

The graph depicted in figure 3.5 shows a two CNN layers encoder, the input is a (None,400,12) which means it take as input a batch of inputs with size (400 time steps and 12 leads) a tensor, the output of this model is two tensors :

- the mean with shape 2 which means a 2D space
- a variance that is assumed as a logarithmic value, a 2D tensor as well

the latent space is 2, this means that the encoder will take every input and project it into a 2D latent space (plan) and the decoder will take a random point from this space because of the term $z_{mean} + exp(0.5 * z_{logvar}) * \epsilon$ and try to decode it back to a valid input (the same input that was projected). Now the latent space dimension is a hyperparameter (a parameter that can be adjusted or tuned to get the best possible results) we can choose the best possible value according to the MMD value, the parameter that gives the minimum MMD will be considered, choosing the value 2 for such parameter will give good results as w'll see later.

For the LSTM-LSTM VAE, the same architecture has been used, which means instead of a CNN layer we use LSTM.

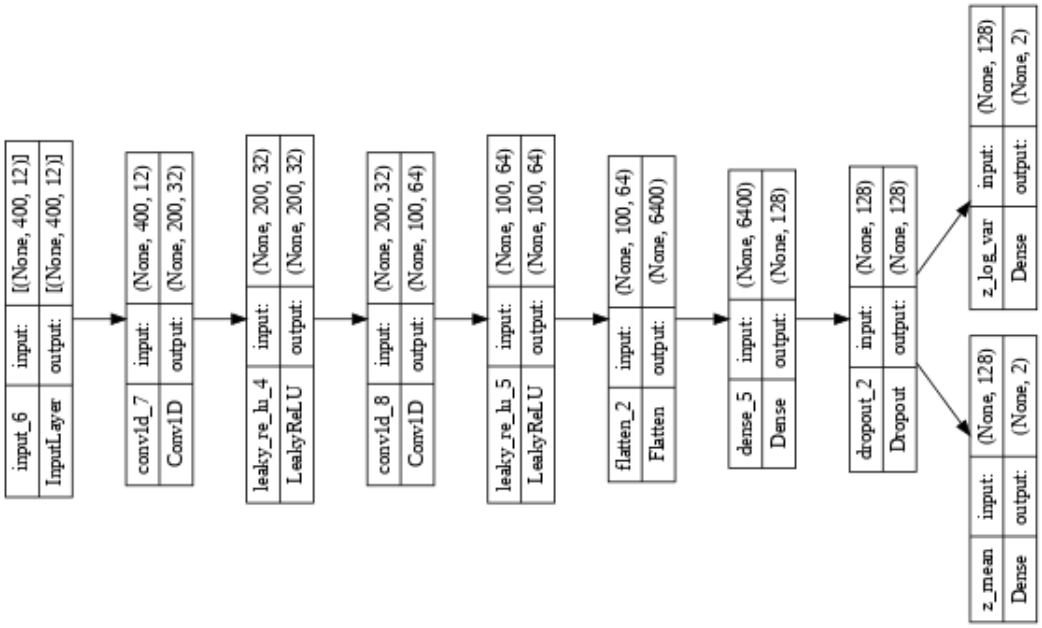


Figure 3.5: CNN encoder

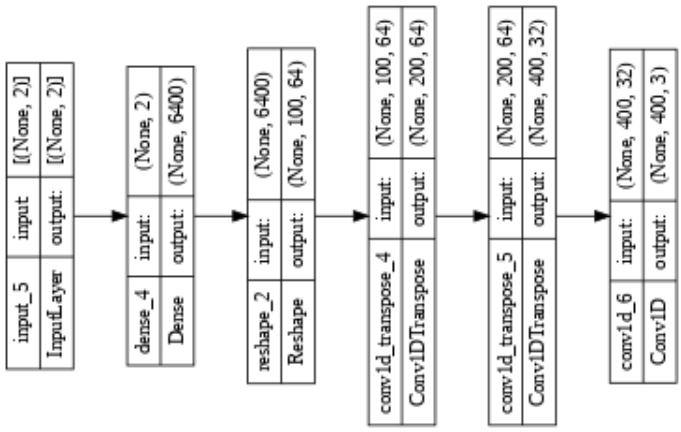


Figure 3.6: CNN decoder

3.3.2 CNN-CNN and LSTM-LSTM GANs

As just for the VAE, a CNN-CNN (respectively LSTM-LSTM) GAN consists of a convolutional neural network (respectively recurrent neural networks) based generator and discriminator.

The discriminator takes as input a batch of tensors and output a single float the is a probability if the input is real, has come from the train dataset or the input is fake.

the generator on the other hand take a random input from a latent space, with dimension 128 in this case, a random noise and try to convert this noise to a data that is similar to the train data, until it learn the latent space, which means every point that the generator takes from the latent space, the generator will try to convert it to a valid input, the feedback loop that helps the generator learn the latent space comes from the discriminator, and we could add to this information a supervision information to help the generator.

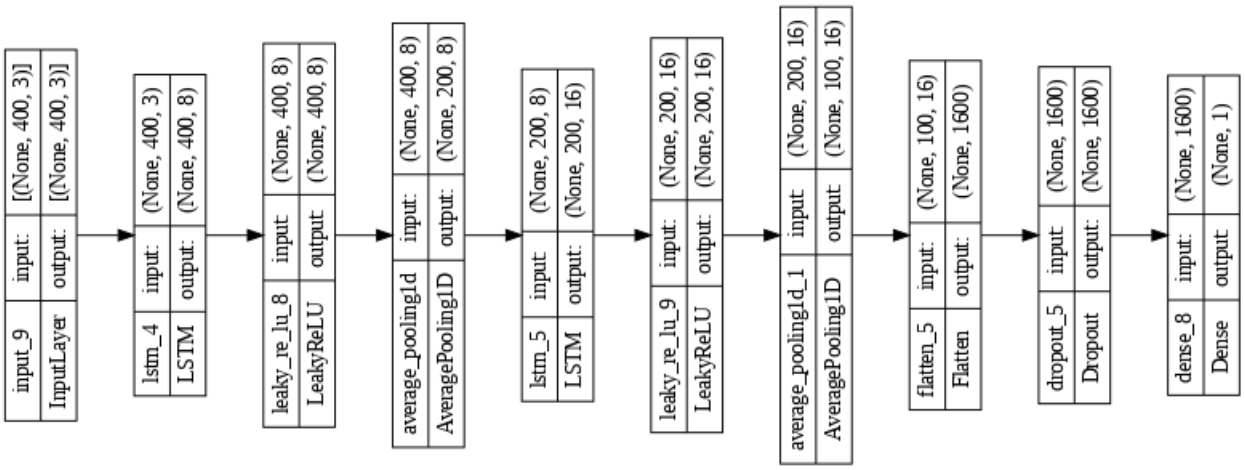


Figure 3.7: RNN discriminator

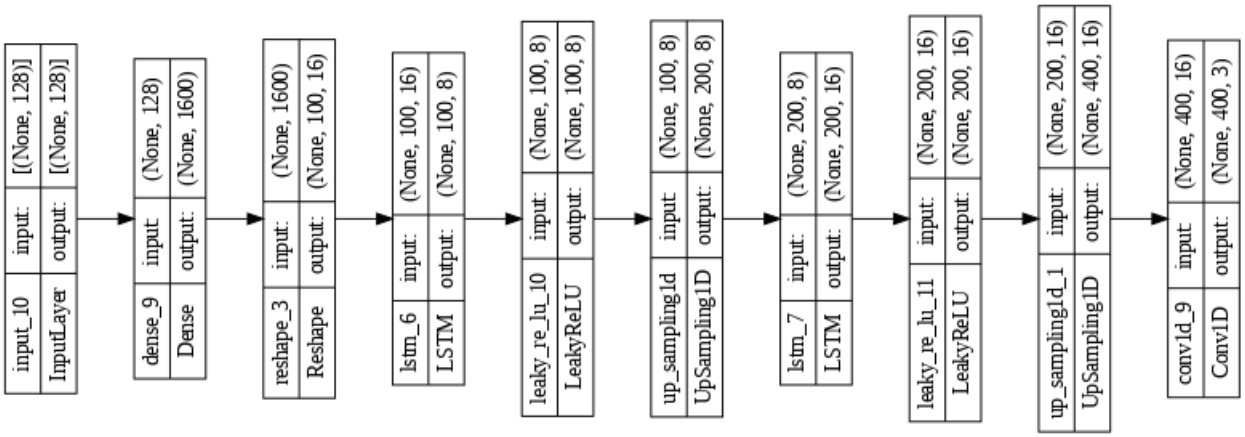


Figure 3.8: RNN generator

3.4 Data generation

3.4.1 Generating with a CNN-CNN VAE

when training starts, the network parameters are initialized randomly, and until epoch 0 which is the first iteration to process all data, the generated data is nothing like a valid ECG.

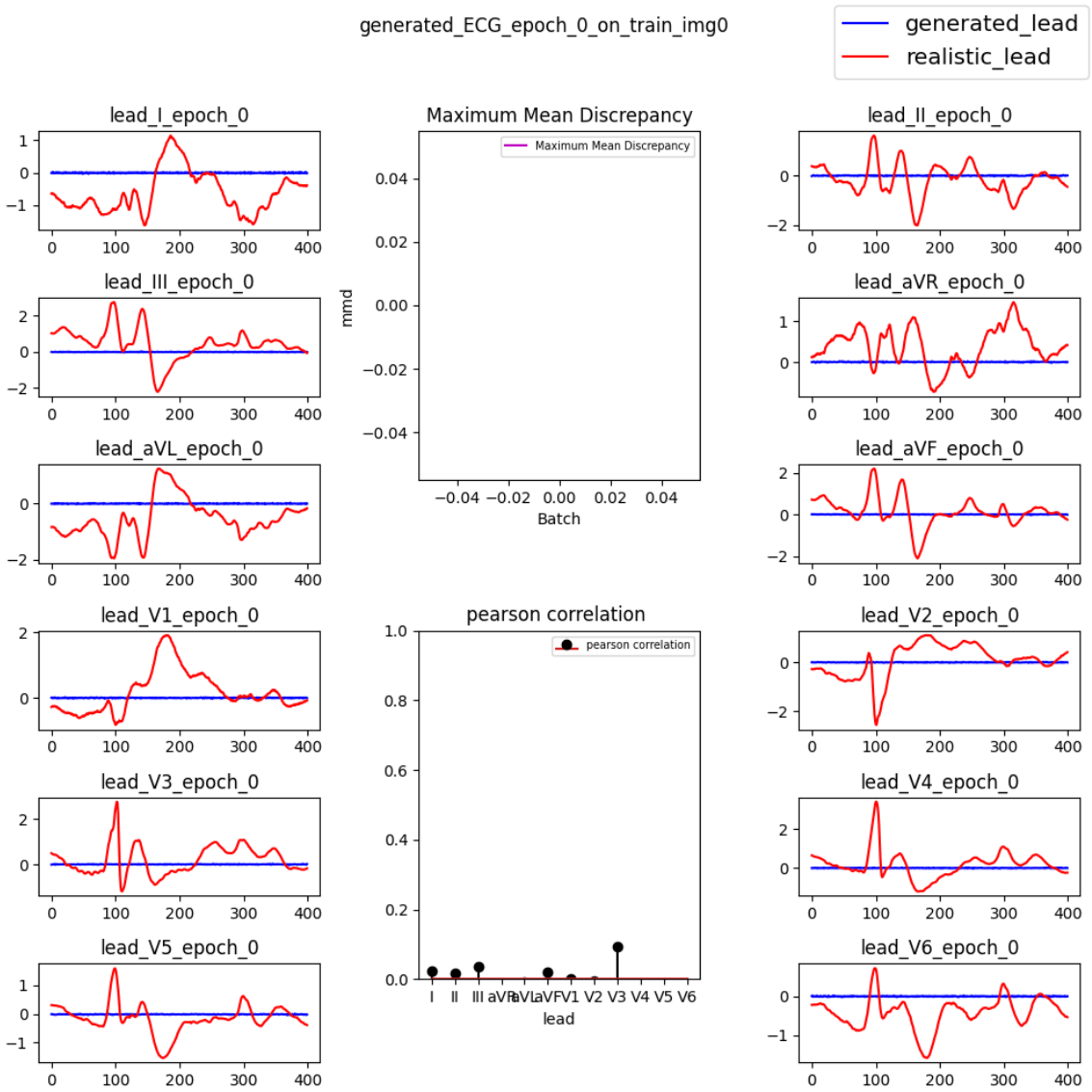


Figure 3.9: generated data on epoch 0

Until epoch 30 the model starts to generate more realistic data, maximum mean discrepancy converges toward 0 ($MMD = 0.0038$) and the Pearson correlation gets closer to

1.

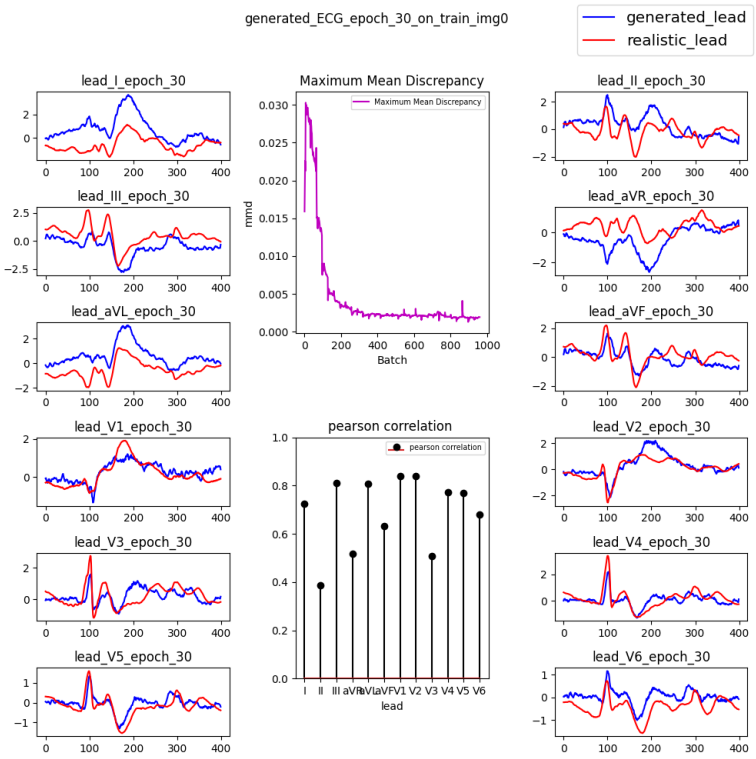


Figure 3.10: generated data at epoch 30

Now we can look at some generated samples and how the latent space looks like if we project the inputs on it, we can also project the random generated samples to see their locations on the latent space figure 3.11.

Figure 3.12 shows a generated window, comparing it to figure 3.3 we can see that the generated data is not very smooth (noisy) and that the QRS-complex is not as sharp as in the train data for some leads, but over all the data looks like an ECG.

3.4.2 Generating with an LSTM-LSTM VAE

The next figures show the generated data from epoch 30 and 60 and the projection of inputs into the latent space.

from MMD plot we can see that MMD is not stable for every processed batch however as the figure 3.15 shows the generated leads are smoother than those generated with a CNN-CNN VAE

The following table shows the results of generation with both VAEs

For the next parts, the LSTM-LSTM VAE will be used.

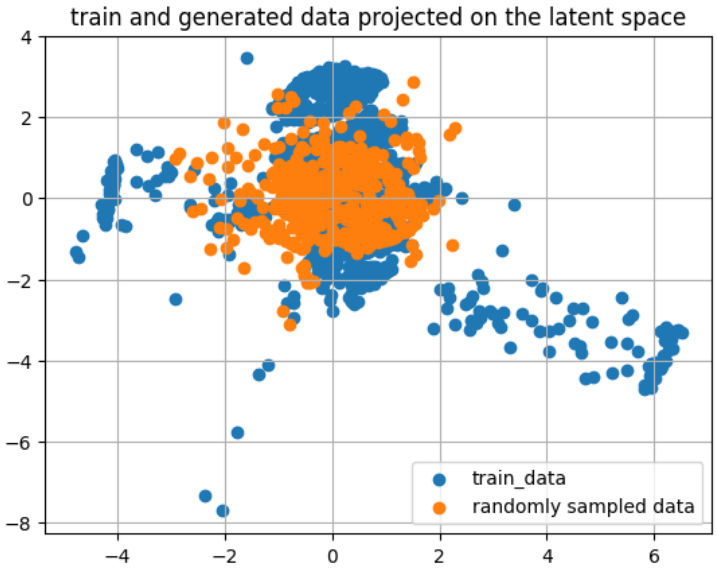


Figure 3.11: train data distribution on the latent space

VAE Architecuture	Encoder parameters	Decoder parameters	mmd
CNN-CNN	829668	214136	0.0038
RNN-RNN	38019	48947	0.002

Table 3.1: LSTM-LSTM and CNN-CNN VAEs trainable parameters

3.4.3 Generating with an CNN-CNN GAN

The most noticeable issue with GANs, is they are hard to train, as opposed to a VAE where we project data as the parameters of a probability distribution and then sample from it, the GAN generator takes noise and convert it to some sort of output, normally gradient descent consists of rolling down in a static loss landscape, with a GAN every step taken down the hill changes the entire landscape a Little which make a GAN a dynamic system that seeks an equilibrium between two forces (gradient ascend of discriminator and gradient descent of the generator) and not a minimum[6], the dynamic of this system can be seen when plotting the discriminator loss.

Just as what we did with the VAE, we can check some generated samples

it's clear that unlike the generation with a VAE where MMD decreases and Pearson correlation increases, a GAN isn't very stable, an instability that can be useful for generation never seen data before or that pushes the GAN toward divergence.

we can also plot the evolution of the generator and discriminator losses

both losses are not stable, and they oscillate, this is a sort of experimental proof of why such models are difficult to train. We can plot a random generated sample and check how

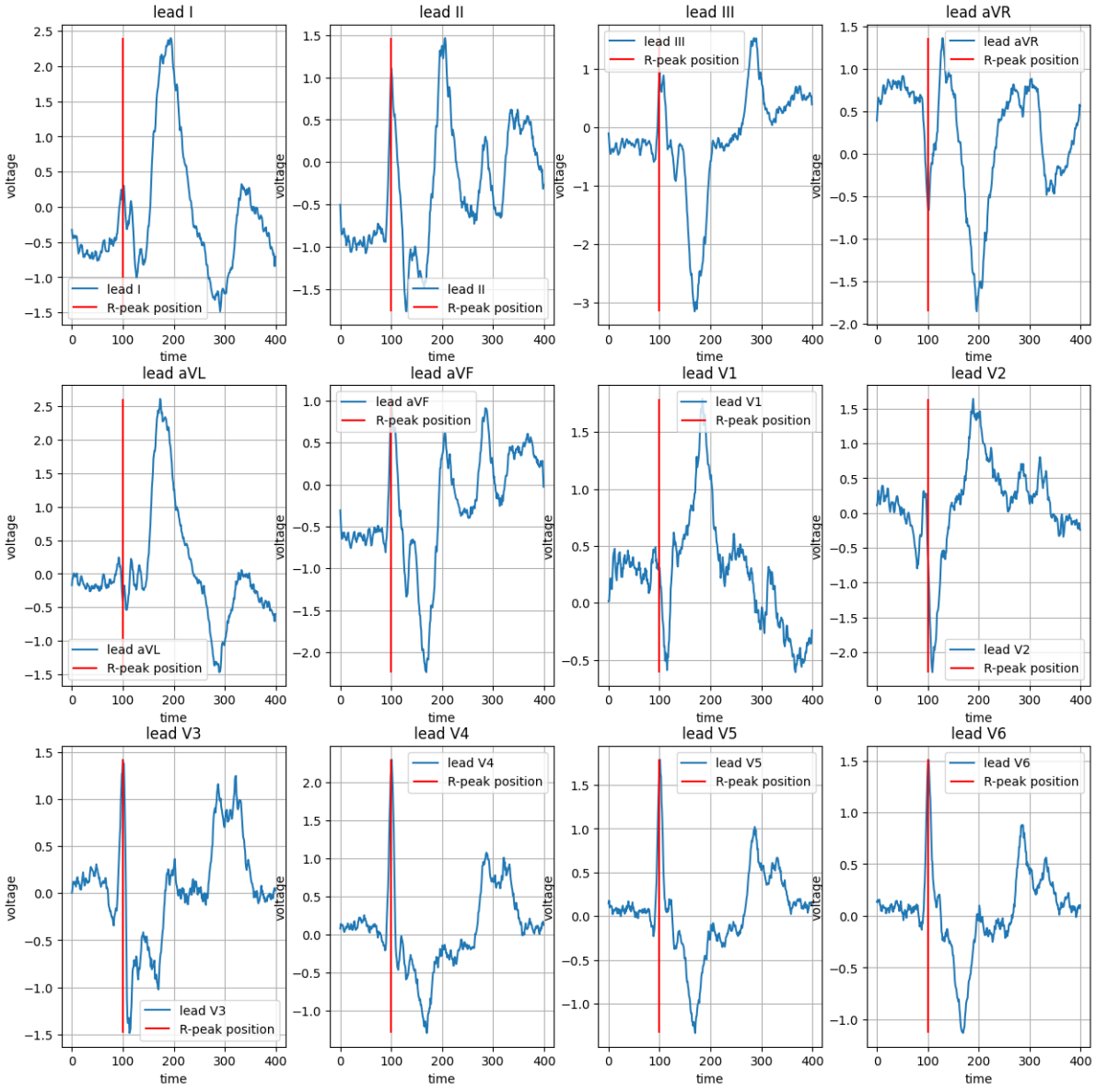
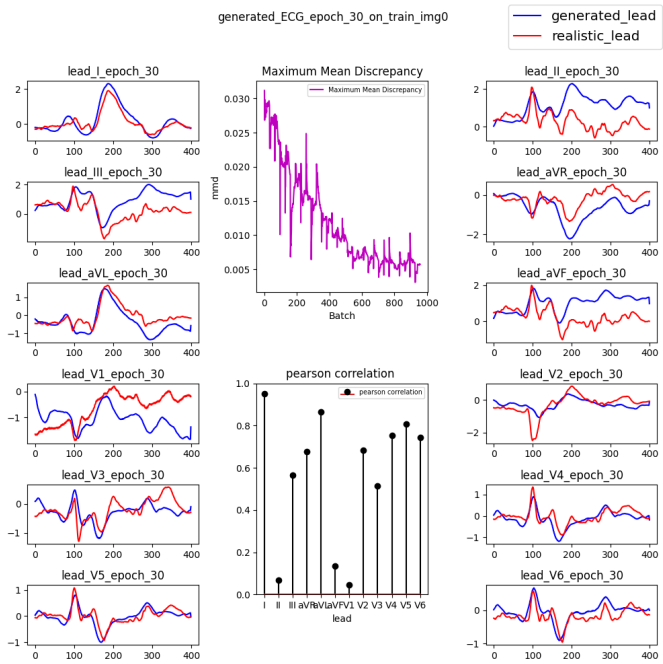


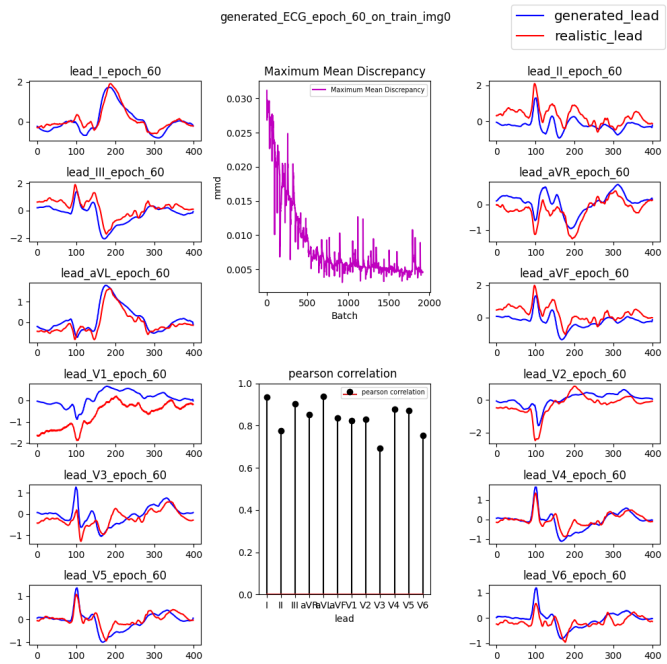
Figure 3.12: generated leads

realistic its leads are.

Although the generated data is acceptable, they contain a lot of artifacts and noise, noise can be helpful to make those data suitable for augmentation, and we can cancel that noise by using an LSTM-LSTM GAN instead.



(a) Generated leads on epoch 30 using LSTM-LSTM VAE



(b) Generated leads on epoch 30 using LSTM-LSTM VAE

Figure 3.13: Generated leads using LSTM-LSTM VAE

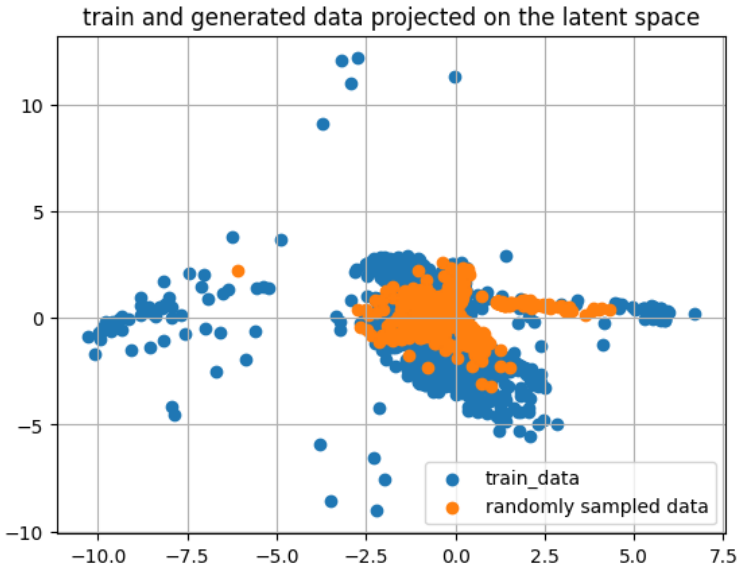


Figure 3.14: data projected on the latent space

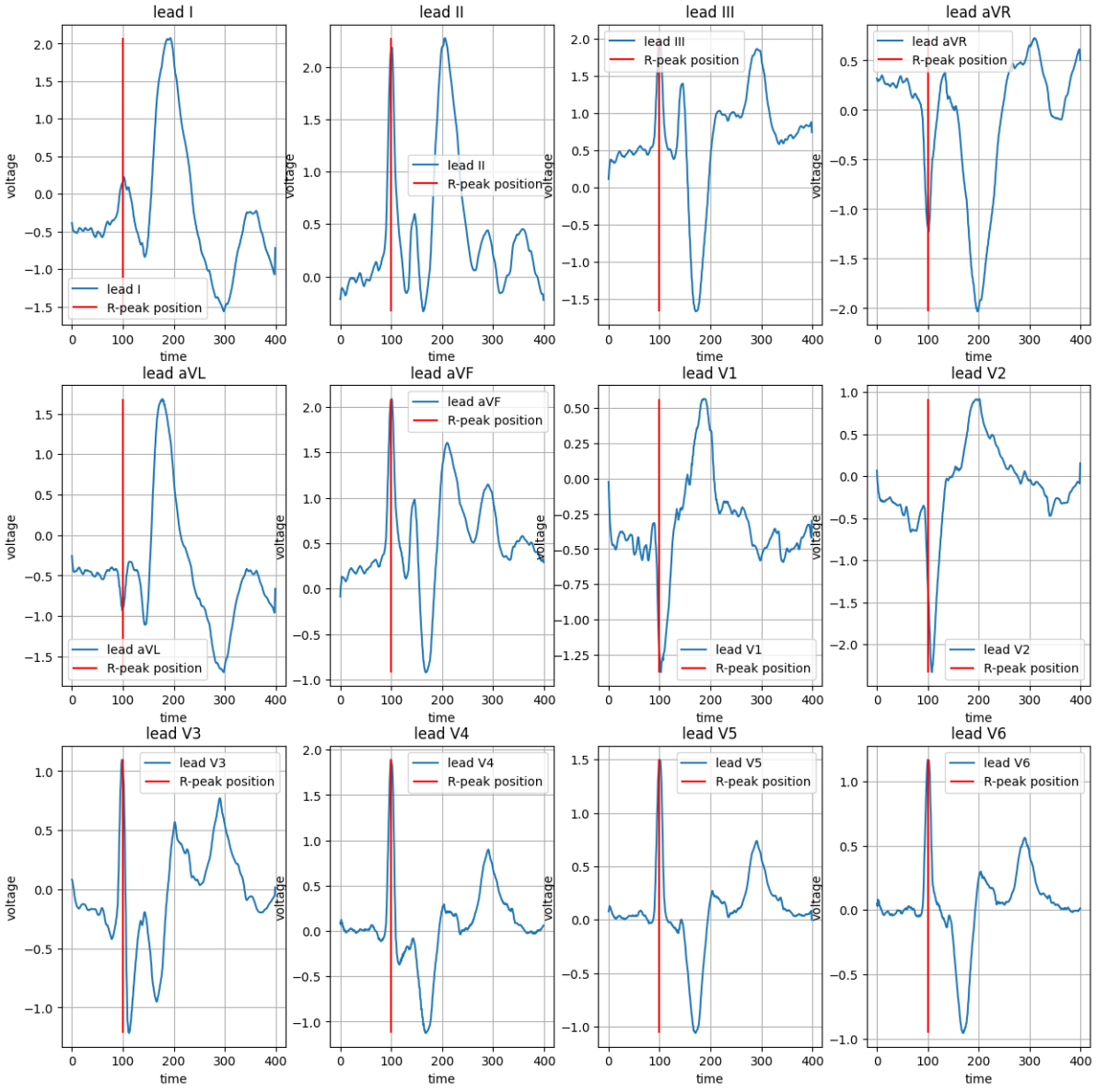
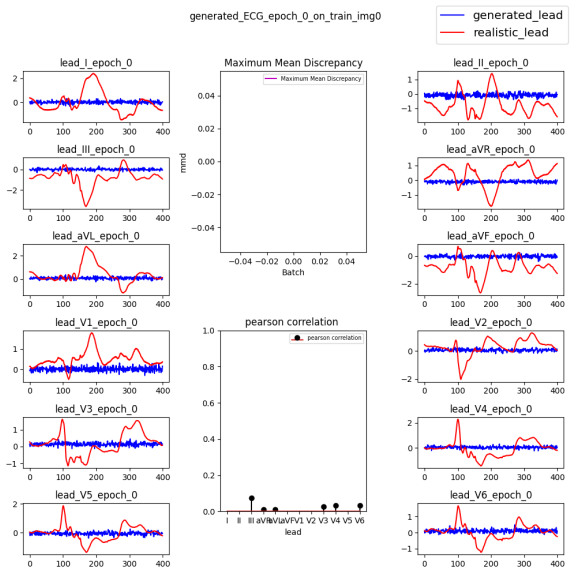


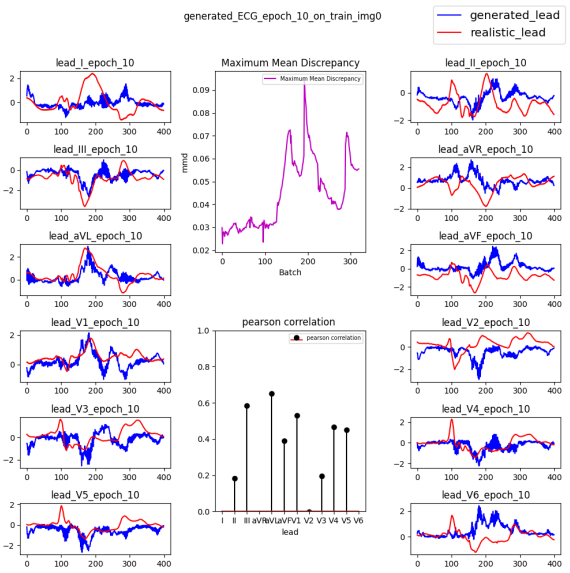
Figure 3.15: generated leads with an LSTM-LSTM VAE

3.4.4 Generating with an LSTM-LSTM GAN

Figure 3.21 depicts some of the generated leads An LSTM-LSTM GAN generates smooth data, but it fails to generate increasing realistic ECG, in a way that our choice is to use a CNN-CNN GAN for ECG generation.

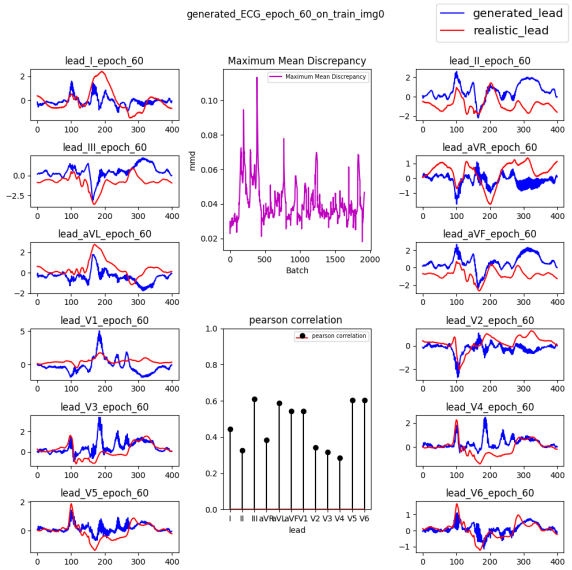


(a) Generated leads on epoch 0 using CNN-CNN GAN

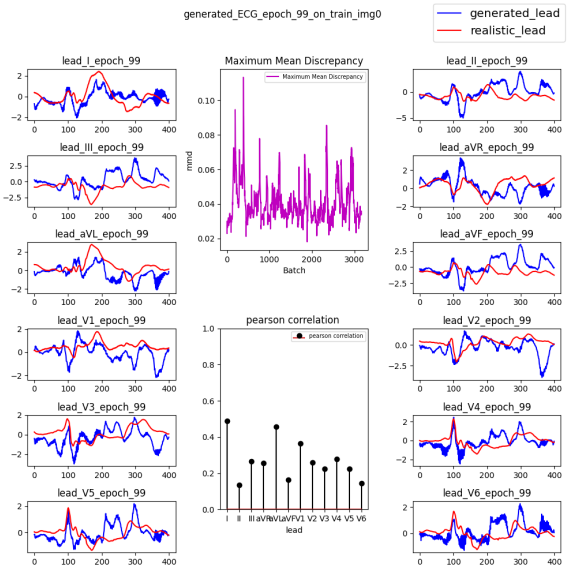


(b) Generated leads on epoch 10 using CNN GAN

Figure 3.16: Generated leads using CNN-CNN GAN



(a) Generated leads on epoch 60 using CNN-CNN GAN



(b) Generated leads on epoch 99 using CNN-CNN GAN

Figure 3.17: Generated leads using CNN-CNN GAN

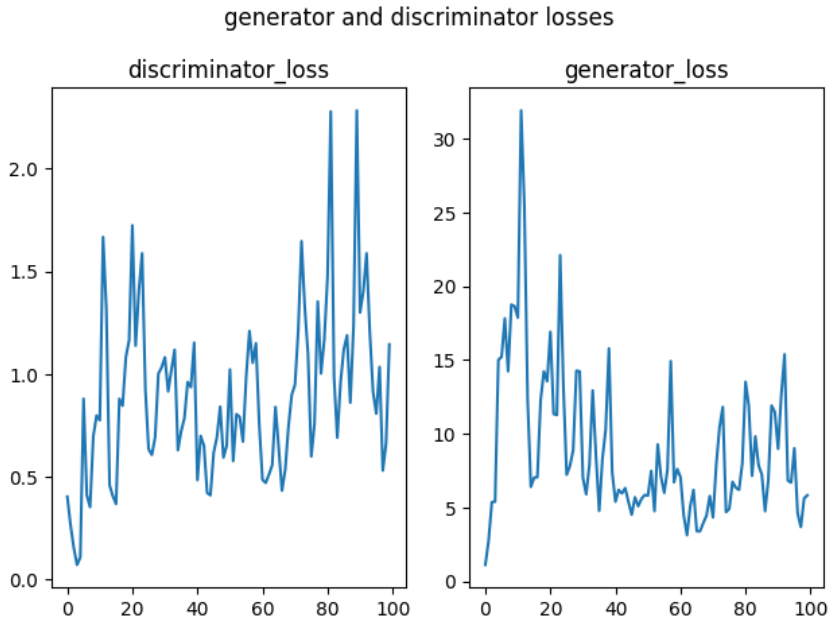


Figure 3.18: generator and discriminator losses

3.5 R-Peak detection without data augmentation

First, we train segmentation models with only the available data from Krug's dataset. as we discussed previously the dataset has a recording of 6 patients, each recording contains a specific number of R-peaks and the total number of extracted samples of size 512 time steps is 3538, the next table summarizes the dataset files.

The model will take as input slices of length 512 time-steps, produces as target a mask, the width of the region that encloses a QRS-complex was chosen to be 39 time steps, where we take 20 time steps before the exact R-peak position and 19 time steps after, this width is a hyperparameter and could be different, since such region encloses a full QRS-complexe as shown in figure 2.20 in the previous chapter it's a valid choice.

the number of trainable parameters of each model are summarized in the next table, the more trainable parameter a model has the more it will take time to produce a prediction during inference (after training and exporting the model).

For Training, the number of leads can change, in this work instead of training on the 12-leads we used a mathematical transformation, (**Kors regression**), that output 3-leads out of the 12-leads, the process is called **Vectorcardiography (VCG)**, the benefits of such transformation are it reduces model complexities, since convolution and other processes will be applied to only 3 instead of 12 signals.

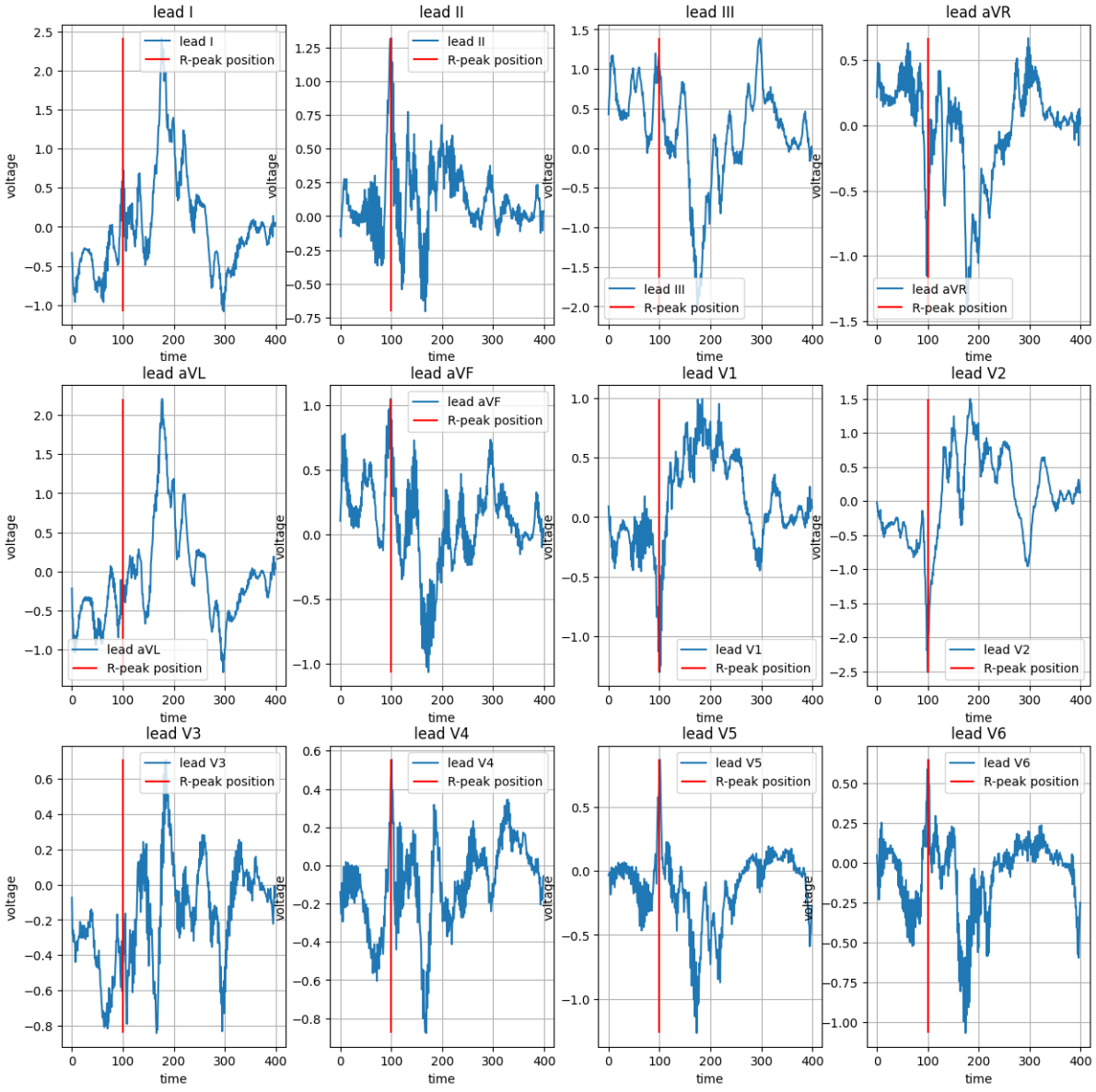
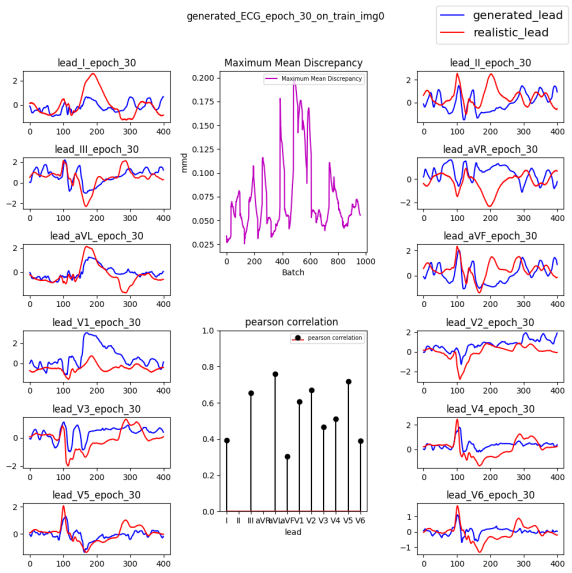


Figure 3.19: a random generated sample

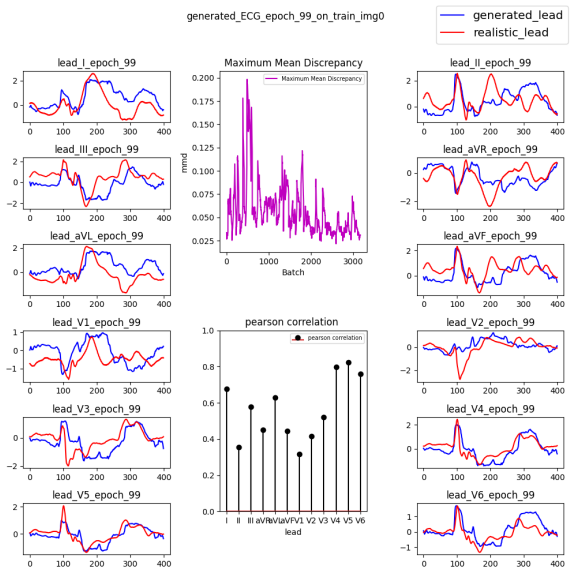
3.5.1 Epsidy's U-Net model results

The results of Epsidy's U-net model are presented in the following table

Epsidy's U-Net model has achieved great results (precession 99.92) in terms of producing only 3 FPs, however FN's are a little bit high, To effectively test this model we used a different dataset **DIJON** [15] that has been recorded under a field strength of 1.5 Tesla, for this we train the model on all recordings and test on DIJON, an example slice from Dijon is shown below.



(a) Generated leads on epoch 60 using LSTM-LSTM GAN



(b) Generated leads on epoch 99 using LSTM-LSTM GAN

Figure 3.20: Generated leads using LSTM-LSTM GAN

file name	length in seconds	number of R-peaks
ECGMRI3T01HF	357.62	400
ECGMRI3T05HF	577.35	659
ECGMRI3T06HF	681.9	689
ECGMRI3T07HF	597.09	688
ECGMRI3T08HF	684.92	773
ECGMRI3T09HF	636.34	887
	3535.22	4096

Table 3.2: recording files

The test results of Epsidy's model on DIJON are presented in the following table.

Although precision is high, the model failed on detecting almost half of R-peaks that exists.

model	total number of trainable parameters
Epsidy's Model	79409
Mini-Xception	51113
Transformer based	33124

Table 3.3: number of trainable parameters of each model

Epsidy's U-NET ,Krug_dataset:[3532 sample,[-20,19] mask width							
fold N~	number of R-peaks	TPs	FNs	FPs	Precision	F1-Score	Recall
1	400	398	2	0	100	99.75	99.5
2	660	653	7	0	100	99.47	98.94
3	689	640	49	0	100	96.31	92.89
4	689	577	112	1	99.83	91.08	83.74
5	773	762	11	2	99.74	99.16	98.58
6	887	885	2	0	100	99.88	99.77
			366	3	99.928	97.6	95.57

Table 3.4: Epsidy's Model results

Epsidy's U-Net model test on DIJON						
number of R-peaks	TPs	FNs	FPs	Precision	F1-score	Recall
1085	609	476	12	98.07	71.4	56.13

Table 3.5: Epsidy's Model DIJON test results

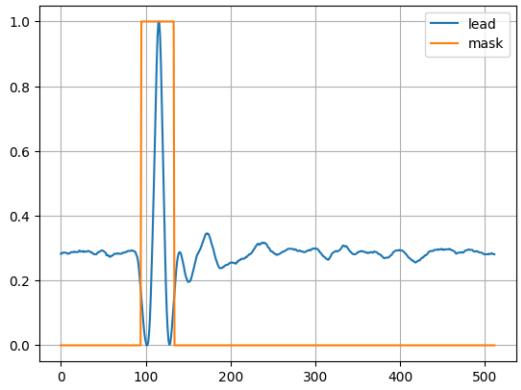


Figure 3.21: a random sample from DIJON dataset

3.5.2 Mini-Xception U-Net like model

The LOOCV results of Mini-xception are presented in the following table, we can see that the FNs are less than that of Epsidy's U-Net, but FPs are higher, which means less MR scan time but higher low quality MR images.

Mini-Xception U-NET like ,Krug_dataset:[3532 sample,[-20,19] mask width							
fold N~	number of R-peaks	TPs	FNs	FPs	Precision	F1-Score	Recall
1	400	400	0	0	100	100	100
2	660	655	5	0	100	99.62	99.24
3	689	661	28	5	99.25	97.57	95.94
4	689	611	78	3	99.51	93.78	88.68
5	773	770	3	7	99.1	99.35	99.61
6	887	878	9	0	100	99.49	98.99
			123	15	99.64	98.3	97.07

Table 3.6: Mini-Xception Model train results

From DIJON test, we can see that the number of FN's is now less than that of U-Net model and F1-score,recall are better.

Mini Xception U-Net like model test on DIJON						
number of R-peaks	TPs	FNs	FPs	Precision	F1-score	Recall
1085	967	118	34	96.6	92.71	89.12

Table 3.7: Mini-Xception U-Net like Model DIJON test results

3.5.3 Transformer based model

Transformer Based Model ,Krug_dataset:[3532 sample,[-20,19] mask width							
fold N~	number of R-peaks	TPs	FNs	FPs	Precision	F1-Score	Recall
1	400	400	0	0	100	100	100
2	660	653	7	0	100	99.47	98.94
3	689	681	8	12	98.27	98.55	98.84
4	689	670	19	1	99.85	98.53	97.24
5	773	770	3	0	100	99.8	99.61
6	887	878	4	0	100	99.77	99.5
			41	13	99.68	99.35	99.02

Table 3.8: Transformer based model, DIJON test results

We can a great reduction in FN's compared to two both previous models, the next table shows test results for DIJON which shows if the model can generalize to different cases or not.

Transformer Based model test results on DIJON						
number of R-peaks	TPs	FNs	FPs	Precision	F1-score	Recall
1085	1022	63	24	97.7	95.92	94.19

Table 3.9: Transformer based Model, DIJON test results

From this table, we can see that the transformer based model that leverage the attention mechanism outperforms other models, and we can conclude that using training data only, those models performs well, hence to be able to test whether generative models are good for data generation or not we will need to reduce the training dataset size, we reduced the total size of the dataset by leaving only 2000 sample for training instead of 3538 and this to test the model on Dijon dataset. for LOOCV we remove from each grouped 5 recordings 1500

sample. We apply this to the transformer encoder based model, the results are shown in the following table for LOOCV and Dijon test.

Transformer encoder based model ,Krug_dataset:LOOCV ,-1500 samples[-20,19] mask width							
fold N~	number of R-peaks	TPs	FNs	FPs	Precision	F1-Score	Recall
1	400	400	0	0	100	100	100
2	660	645	15	1	99.85	98.78	97.73
3	689	567	122	47	92.35	87.03	82.29
4	689	535	154	139	79.38	78,51	77.65
5	773	559	214	50	91.79	80.9	72.32
6	887	846	41	48	94.63	95	95.38
			546	285	93	92.34	87.56

Table 3.10: Transformer based model LOOCV results with reduced dataset size

From the statistics shown in table 3.10, compared to training with the full dataset the performances largely drops, which is normal, training a model on a small dataset for a given number of epochs will make the model overfits on train data producing low performances on validation or test for never-seen data (large number of FNs), for testing on Dijon dataset we have the following results.

Transformer encoder based model testing on Dijon,samples[-20,19] mask width						
number of R-peaks	TPs	FNs	FPs	Precision	F1-score	Recall
1085	899	186	16	98.25	89.9	82.86

Table 3.11: Transformer based model test results on Dijon dataset

3.6 R-peak detection with data-augmentation

To be able to show if generative models are effective for data augmentation if we have a few training samples, we use the previous validation scheme, first we use an LSTM-LSTM VAE to generate 1500 sample, we add them to the reduced krug's dataset (2000 sample) for training, similarly for a GAN

3.6.1 Data augmentation with an LSTM-LSTM VAE

The statistics shown in table 3.12 proves that a VAE can be used to improve performance if the dataset has a few numbers of training samples, we've got an average of 98 for precision,

LOOCV + VAE data augmentation							
fold N~	number of R-peaks	TPs	FNs	FPs	Precision	F1-Score	Recall
1	400	400	0	0	100	100	100
2	660	657	3	2	99.7	99.62	99.55
3	689	653	36	42	93.96	94.37	94.78
4	689	659	30	6	99.1	97.34	95.65
5	773	767	6	9	98.84	99.03	99.22
6	887	878	9	7	99.21	99.1	98.99
			84	66	98.46	98.24	98.03

Table 3.12: LOOCV results with data augmentation using an LSTM-LSTM VAE

F1-score and recall which is lower by 1% compared to training with the whole dataset but proves the capability of a VAE for data augmentation, the number of generated samples should be always less than the total size of the train set, otherwise, since those models learn the statistical distribution of the original data and samples data that are in between, they will overfit.

Transformer encoder based model testing on Dijon,samples[-20,19] mask width						
number of R-peaks	TPs	FNs	FPs	Precision	F1-score	Recall
1085	841	244	11	98.71	86,83	77,51

Table 3.13: Test on Dijon dataset with VAE data augmentation

3.6.2 Data augmentation with a CNN-CNN GAN

From the statistics presented in the previous table we can see that a GAN was a little worse than a VAE, the main reason was the luck of large data as well as the dataset used for training doesn't have a lot of statistical diversity since the subjects are has no heart illness and since the MR environment is the same environment.

Testing on Dijon holds worst results than the results of data augmentation with a VAE,

3.7 Conclusion

In this chapter I have presented the generation process and generated data quality for both VAEs and GANs as well as the models that been tested and trained for the R-peak detection, models based on attention proved to be superior in such task as they proved their superiority in language tasks, we have also presented the effect of data augmentation using VAEs and GANs, in which we got better results using a VAE.

LOOCV + GAN data augmentation							
fold N~	number of R-peaks	TPs	FNs	FPs	Precision	F1-Score	Recall
1	400	400	0	0	100	100	100
2	660	652	8	1	99.85	99.32	98.79
3	689	670	19	30	95.97	96.47	97.24
4	689	596	93	25	95.97	90.99	86.5
5	773	754	19	11	98.56	98.05	97.54
6	887	854	33	0	100	98.1	96.28
			172	67	98.39	97.155	96.05

Table 3.14: LOOCV results with data augmentation using an CNN-CNN GAN

Chapter 4

Conclusion and prospects

In this work we attempted to train deep generative models for data augmentation purposes, and it was the main subject of the internship, due to the lack of publically available datasets from the medical filed, data that are under restrictions for privacy precautions makes it difficult to apply scientific and technological advancements in certain fields especially deep learning, a field that relies on large datasets, although generative models are nothing more than deep neural networks, they certainly obey the same rule, if small set of data is provided they can be of such large use, the work was subdivided into two parts:

we tried first to develop a model that outperforms already exciting models that have been tested for the task of R-peak detection, where I developed a self attention based model that uses the vanilla transformer with some additional tweaks to be adapted for signals rather than text, the attention mechanism that is the core of the transformer seemed to work well for this task with the use of a lightweight architecture I was able to achieve good results.

As for the second part, we wanted to test weather variational auto-encoders and generative adversarial networks are able to generate never seen data using only a few training data, the metric that has been used by many researches was the maximum mean discrepancy, although the value of this probabilistic distance can give a hint if the generated data is realistic or not the generated data should be used in real cases to assess the usefulness of such models were we have proved that such model can be used with a constraint that they can't generate a large amount of diverse data.

A future work for the generation part was to use sequence to sequence models and to treat time series like language, thus a full transformer architecture could be applied to such problems, the only constraint is that an expert in ECG annotation needs to annotate the generated sequences which is time-consuming, but I believe that is more effective.

Bibliography

[1] R. Schmidt and G. Thews, *Human Physiology* (Springer-Verlag, 1989), ISBN 9783540194323.

[2] C. Chernecky, *ECGs & the Heart*, Real world nursing survival guide (W.B. Saunders, 2002), ISBN 9780721690360.

[3] M. PhD and D. Mann, JACC: Basic to Translational Science **1**, 95 (2016).

[4] C. Westbrook, C. Roth, and J. Talbot, *MRI in Practice* (Wiley, 2011), ISBN 9781118273869.

[5] J. Passand Krug, G. Rose, G. Clifford, and J. Oster, Journal of cardiovascular magnetic resonance : official journal of the Society for Cardiovascular Magnetic Resonance **15**, 104 (2013).

[6] F. Chollet, *Deep Learning with Python* (Manning, 2017), ISBN 9781617294433.

[7] A. Burkov, *The Hundred-Page Machine Learning Book* (Andriy Burkov, 2019), ISBN 9781999579517.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, *Attention is all you need* (2017), 1706.03762.

[9] O. Ronneberger, P. Fischer, and T. Brox, *U-net: Convolutional networks for biomedical image segmentation* (2015), 1505.04597.

[10] Y. Bengio and Y. Lecun (1997).

[11] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, *Generative adversarial networks* (2014), 1406.2661.

[12] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, Journal of Machine Learning Research **13**, 723 (2012), URL http://jmlr.org/papers/v13/gretton12a.html.

[13] F. Chollet, *Xception: Deep learning with depthwise separable convolutions* (2017), 1610.02357.

[14] J. W. Krug Passand, *Influence of the mhd effect on 12-lead and 3-lead ecgs recorded in 1t to 7t mri scanners* (2021).

[15] M. Mehri, G. Calmon, F. Odille, and J. Oster, Sensors **23** (2023), ISSN 1424-8220, URL `https://www.mdpi.com/1424-8220/23/4/2288`.